CS336 · Spring 2026 · Lecture 05

GPU：从数据搬运到 FlashAttention 的中文自学增强

如何阅读这份自学增强讲义

原始英文幻灯片完整保留；每组原页之后紧接对应中文讲解。书签可跳转到原页与讲解。页码中的“原页”始终指原始 PDF 页码。

中文部分为自学补充；课堂转述、原论文结论与编者推导应按文中标记区分。第三方保存的字幕仅用于检索定位，不等同于教师审校逐字稿。

学习目标：读完本讲，应能区分线程、warp、block 与 SM；为简单算子计算 FLOPs、读写字节和算术强度；解释低精度、融合、重计算、合并访存与分块各自减少了什么成本；从稳定 softmax 推导在线更新，最后手算一个不保存完整注意力矩阵的前向过程。原始 55 页完整保留，中文解释紧邻对应原页，数值例子及未标课堂的推导均为编者补充。

先修：矩阵乘法 $C[i,j]=\Sigma_k A[i,k]B[k,j]$、链式法则、softmax、二进制浮点数的基本概念，以及上一讲的 Q/K/V。无需先会 CUDA。阅读主线是“工作如何分配 → 数据放在哪里 → 实际瓶颈是什么 → 如何重排计算”。先跟着每个小例子算出结果，再看原图，就能把硬件结构与算法设计连接起来。

版本与来源：以 Stanford CS336 Spring 2026 第 5 讲原 PDF 为基准。课堂补充来自公开录像的第三方缓存字幕，时间戳仅用于定位，不作为教师审校逐字稿；术语与公式另对照原论文和官方文档。原稿中的单位倒置、格式名称混用等保留在原页，只在中文解释中辨明。末尾给出来源和带解答自测。硬件数字、速度比例与实验曲线均有具体设备和工作负载条件，不视为普适定律。

Lecture 5

GPUs

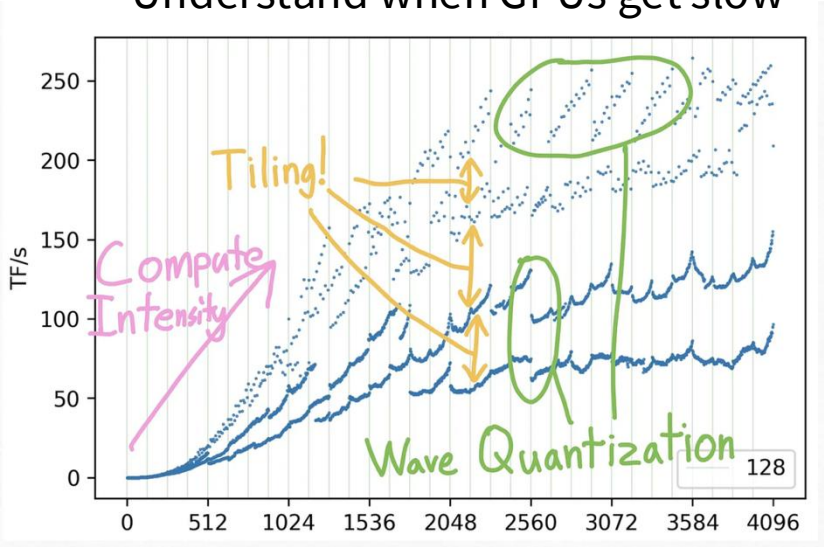
CS336

Tatsu H

Outline and goals

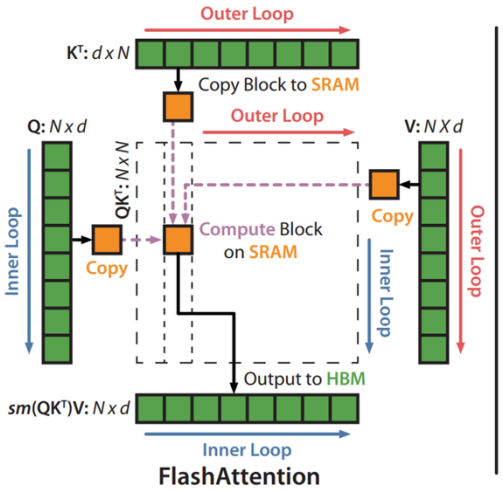
- Make CUDA and GPUs less magic

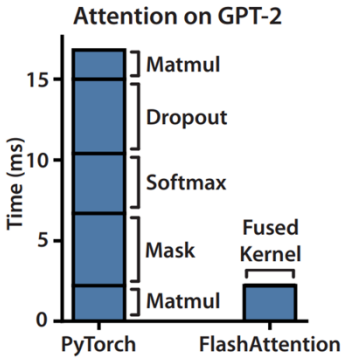
Understand when GPUs get slow



https://www.thonking.ai/p/what-shapes-do-matrix-multiplications

Understand how to make fast algorithms

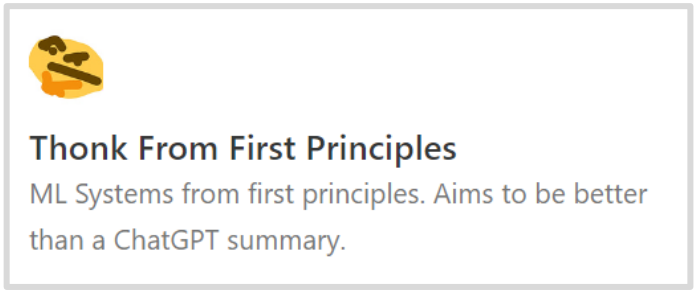


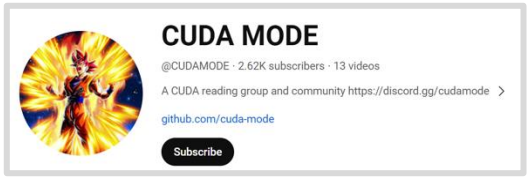


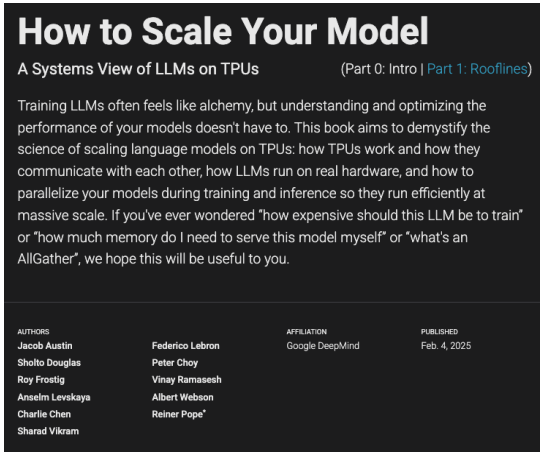
Dao et al, Flash Attention

Before we start..

Substantial credit goes to a few sources that I'd like to highlight..







Horace He's blog

CUDA Mode group

TPU (and now GPU) book (!!)

And other sources including https://nichijou.co/, https://jonathan-hui.medium.com/

第 5 讲 · 原页 1–3 · 中文讲解

本讲要解决的问题与读图方法

从“买更快 GPU”到“让 GPU 真正工作”

本讲关心的是同一个数学计算为什么会有完全不同的耗时。GPU 标称的每秒浮点运算量是上限：如果计算单元一直等待数据，或者工作太少、分配不均，就无法接近它。第 2 页左图横轴是矩阵规模、纵轴是实测吞吐；曲线总体上升又突然跌落，说明性能不是只由乘加次数决定。右图的 FlashAttention 将同一注意力公式换一种计算和存储顺序，便能显著改变时间及显存占用。先记住这两个问题，后面逐一解释。

来源如何使用

第 3 页是教师致谢，帮助辨认图表出处。原稿本身和课堂内容规定本讲范围；CUDA 官方文档用于核对执行与内存语义，FlashAttention 和 online softmax 论文用于核对算法。外部博客中的某张硬件曲线只代表其测量场景。阅读时把“数学等价”“硬件近似模型”“具体实验测量”分开：数学等价需要推导，性能预测需要说明假设，最终速度需要测量。

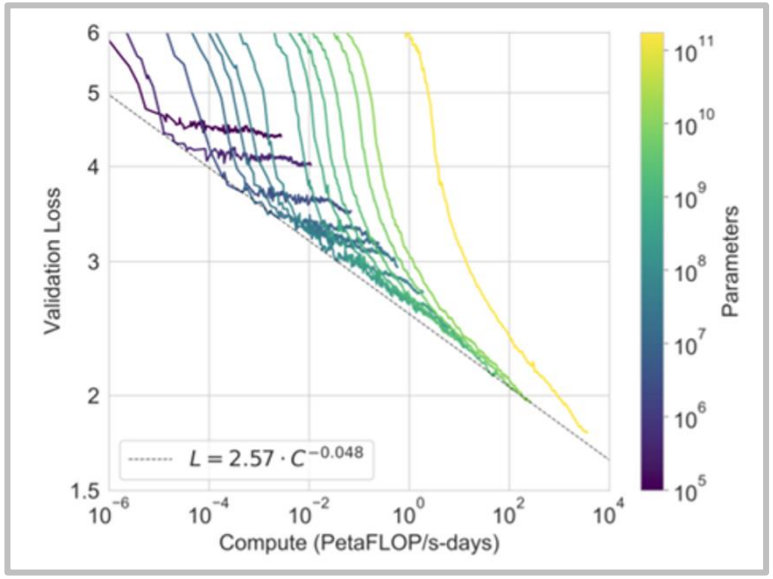
本节参考（完整书目见讲义末尾）：[course] · [slides]

Organization today:

- **Part 1**: GPUs in depth – how they work and important parts
- **Part 2**: Understanding GPU performance
- **Part 3**: Putting it together – unpacking FlashAttention

Setting the stage: compute leads to predictable perf

Often times, compute leads to predictable performance gains for language models



Kaplan et al, Neural Scaling Laws

Faster hardware, better utilization, improved parallelization alone can drive progress (for now..)

第 5 讲 · 原页 4–5 · 中文讲解

三段主线：硬件、性能、算法

为什么系统优化影响模型训练

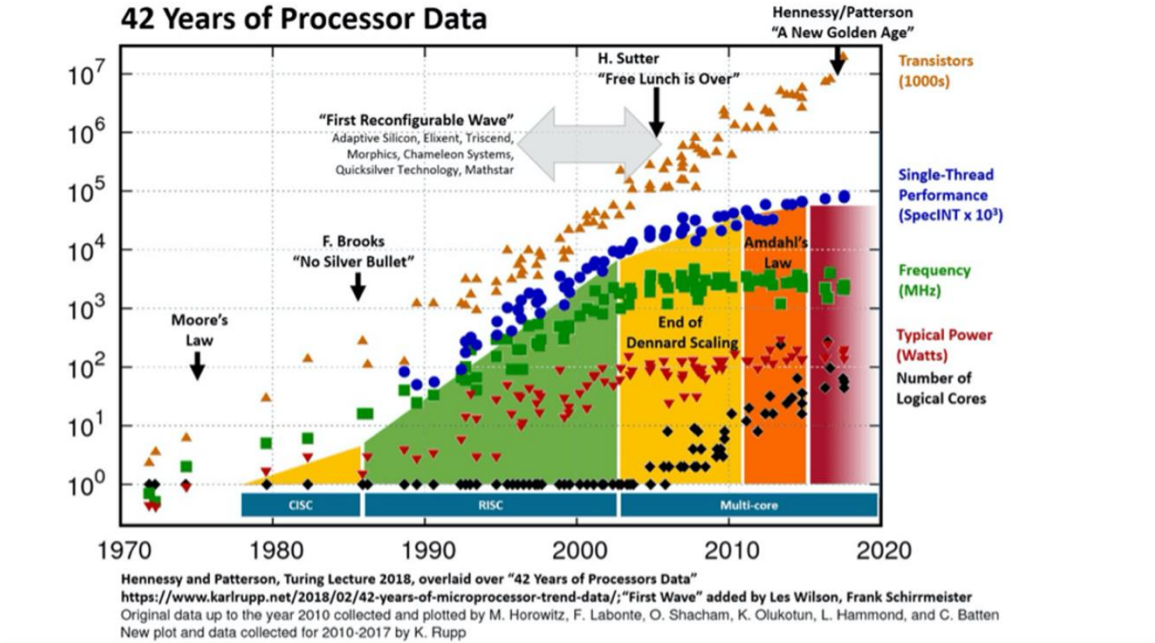
第 4 页分为硬件结构、性能技巧、FlashAttention 三部分。第 5 页的 scaling law 图展示训练计算量与损失之间的经验关系，横轴计算预算增加时，合适模型与数据配比下的损失下降。它不表示给任意模型无限重复训练都会按同一规律改善，也不保证下游能力随每个 FLOP 单调提高。系统优化的直接作用是：给定时间和机器资源，完成更多有用计算；是否转化为模型收益还取决于数据、优化和模型容量。

编者补充：把预算写成等式

可用预算近似写成 C = P_peak × u × t，其中 C 是已完成的有用浮点运算数，P_peak 是所选精度与硬件路径的峰值 FLOP/s，u 是相对该峰值的有效利用率，t 是时间。若同一工作负载的 u 从 0.25 提升到 0.50，在其他条件相同时可将计算时间减半。这个算式解释为何“算法算得少”和“每秒算得快”都重要，但不能用增加无效运算来美化利用率。若一次优化 FLOPs 增多而耗时变短，重计算就是本讲会出现的实例。

本节参考（完整书目见讲义末尾）：[slides]

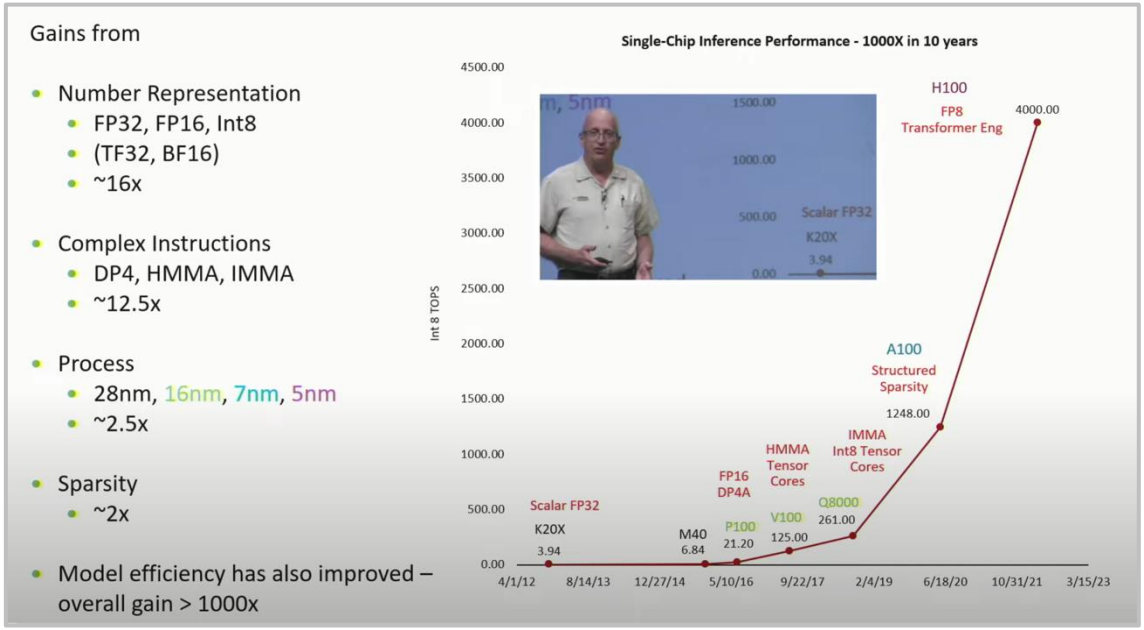
How do we get compute scaling? Early on – Dennard scaing



But the traditional form of scaling (*Dennard scaling*) from 1980-2000s has tapped out.

.. How do we feed LLMs' insatiable appetite for compute?

Parallel scaling continues



Bill dally, HotChips keynote

Parallel scaling with GPUs has scaled > 1000x in 10 years.

There is no LLM scaling without GPU scaling

第 5 讲 · 原页 6–7 · 中文讲解

从 Dennard 缩放到并行吞吐

芯片更密不等于单线程永远更快

第 6 页把晶体管数量、频率、功耗、核心数等历史曲线放在一起。Dennard 缩放的理想图景是晶体管尺寸缩小，同时降低工作电压，让单位面积功耗大致可控，因而提高频率。电压难以继续同比下降后，单纯提高频率会受到散热与能耗限制。图中不同曲线在不同时代分离，不能把 Moore 式晶体管数量增长直接等同于单线程速度增长。

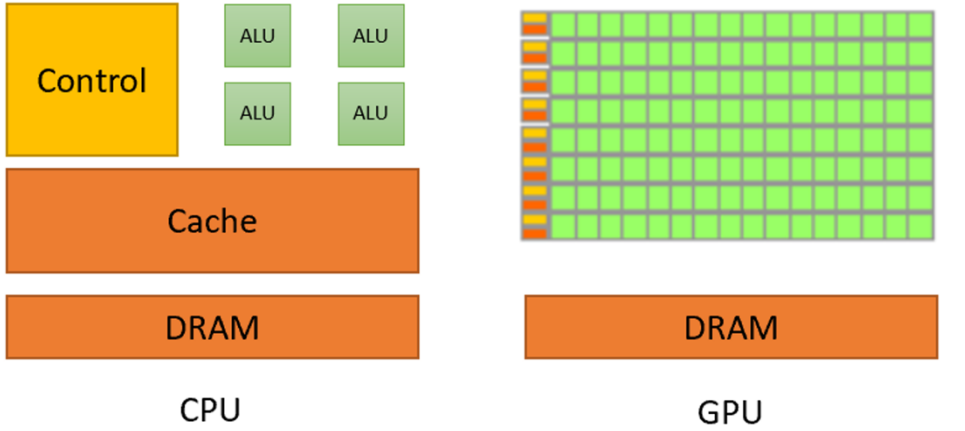
并行是另一条增长路线

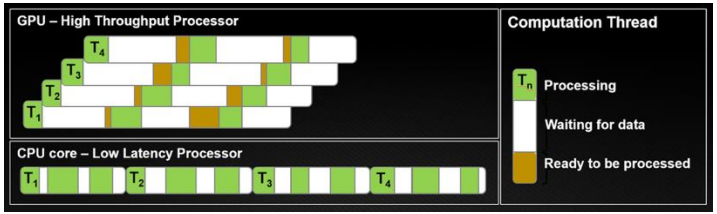
第 7 页将性能增长分解为数值表示、专用运算、工艺等因素。更多小运算单元可以同时处理不同数据，低精度也能让相同面积或带宽服务更多运算；这些因素共同形成图示的历史增益，“十年千倍”不是仅增加 SM 数量的结果。编者例子：有 1000 个互相独立的乘法时，100 个执行单元理想上约 10 轮完成；若后一个结果必须依赖前一个，则不能这样并行。语言模型的巨大矩阵乘法恰好提供大量独立乘加，但归约、同步和数据传输会限制实际收益。

本节参考（完整书目见讲义末尾）：[slides]

How is a GPU different from a CPU?

CPUs optimize for a few, fast threads while GPUs optimize for many many threads





Many tiny compute units (ALUs).
Much less support for branching (control, cache)

CPUs optimize for latency (each thread finishes quickly)
GPUs optimize for throughput (total processed data)

https://developer.nvidia.com/blog/cuda-refresher-reviewing-the-origins-of-gpu-computing/

Anatomy of a GPU (execution units)



SM



GA100 Full GPU with 128 SMs

Each SM further contains many **SPs (streaming processor)** that can execute ‘threads’ in parallel

GPUs have *many* **SM (streaming multiprocessors)** that independently execute ‘blocks’ (jobs).

第 5 讲 · 原页 8–9 · 中文讲解

CPU、GPU 与 SM：吞吐来自哪里

读懂结构示意图

第 8 页 CPU 图把较大面积给控制逻辑和缓存，强调尽快完成少量复杂线程；GPU 图把更多面积给大量算术单元，强调单位时间处理很多相似工作。这是设计倾向，不表示 CPU 不能并行或 GPU 不能执行分支。延迟是单项工作从开始到结束的时间，吞吐是单位时间完成的工作数量；很多慢一点的工作并发运行，也可能获得更高总吞吐。

从软件任务映射到硬件

第 9 页大芯片中的重复小块是 SM，即 streaming multiprocessor。每个 SM 有执行单元、调度资源、寄存器与近端存储，负责运行分配过来的线程块。SP 或 CUDA core 表示其中一种标量运算资源，不能把一个线程永久绑定到一个 SP，也不能把 CUDA core 数直接当作独立 CPU 核数比较。线程是程序视角的执行实例，warp 是调度相关的线程分组，SM 是承载许多线程的硬件。

编者补充：隐藏等待而非消除等待

若一个 warp 等待内存，SM 可以选择另一个已就绪 warp 发射指令。这让昂贵运算单元持续工作，却不使那次内存读取本身变快。必须有足够多独立且驻留的工作才能隐藏延迟；寄存器或共享内存使用过多，会限制驻留工作数。这为后面的“大 tile 有更多重用，却不总是更快”埋下伏笔。

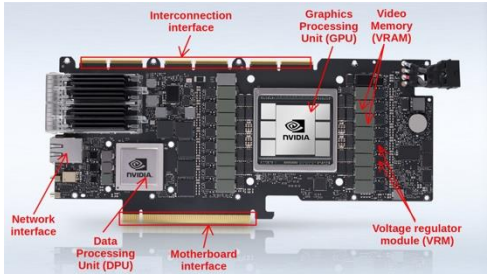
本节参考（完整书目见讲义末尾）：[slides] · [cuda]

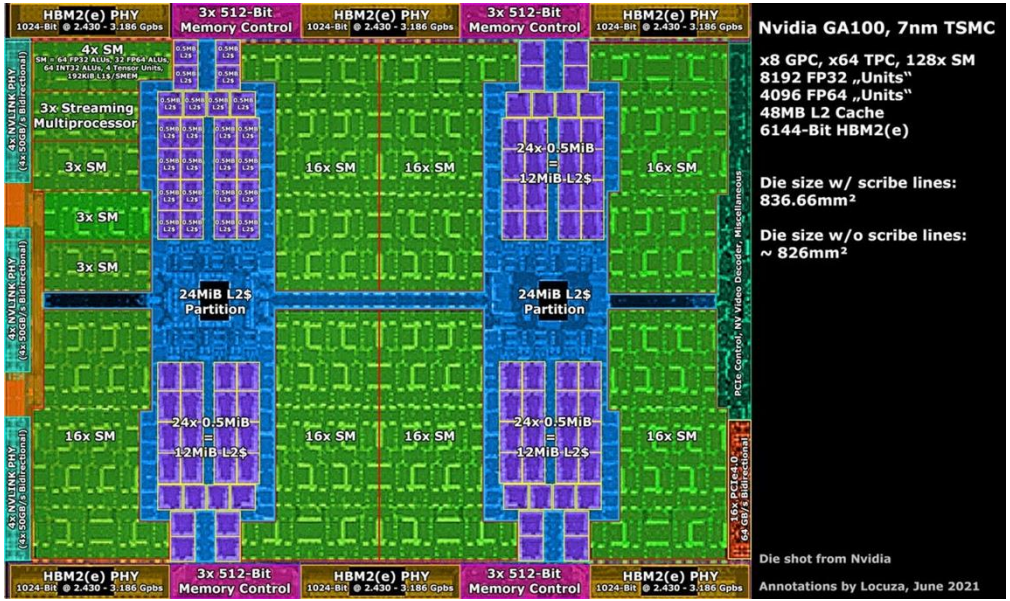
Anatomy of a GPU (memory)

The closer the memory to the SM, the faster it is – L1 and shared memory is *inside* the SM. L2 cache is on die, and global memory are the memory chips next to the GPU

TABLE IV
THE MEMORY ACCESSES LATENCIES

Memory type	*CPI (cycles)*
Global memory	290
L2 cache	200
L1 cache	33
Shared Memory (ld/st)	(23/19)





SRAM (shared/cache memory) is much more expensive (100x) but ~ 8x faster than DRAM (Global memory)

第 5 讲 · 原页 10–10 · 中文讲解

内存层级：容量、延迟、带宽是三个问题

沿着图从芯片内部读到外部

寄存器保存线程的标量和累加器；共享内存位于 SM 附近，由同一 block 的线程显式协作使用；L1/L2 cache 由缓存机制自动复用数据；global memory 通常由 GPU 外围的高带宽 DRAM 提供。图中 SRAM 与 DRAM 的位置和体积差异，是“小而快”与“大但远”的直观来源。容量回答能放多少，延迟回答一次请求多久回来，带宽回答持续传输每秒能搬多少，三者不能混用。

编者补充：为什么多一层会有用

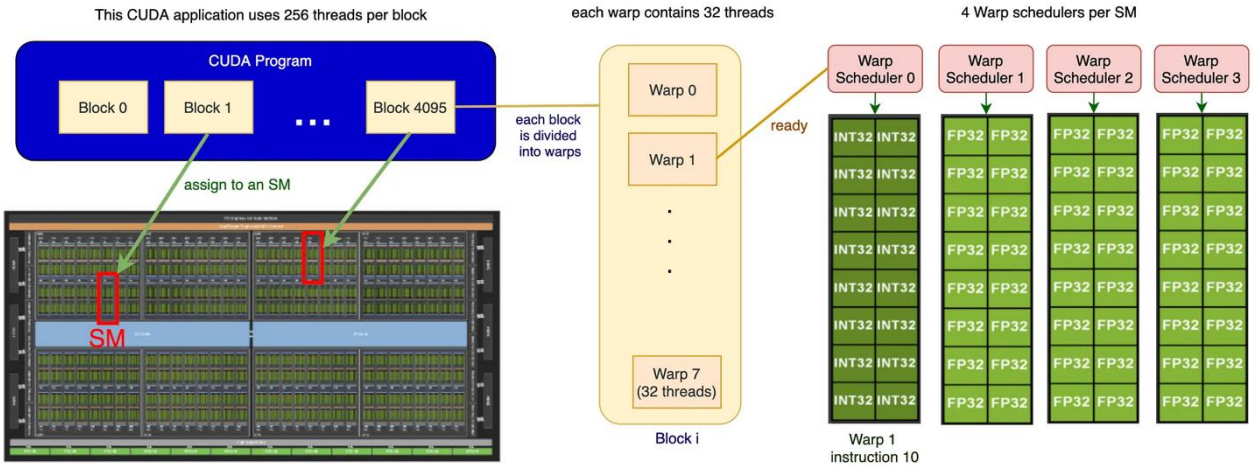
假设一个 FP32 数值供 32 次乘法使用。每次都从 HBM 取，逻辑请求可达 32×4=128 B；先搬入共享内存一次，之后从近端复用，只需一次 4 B 的 HBM 输入，另有共享内存流量。实际硬件还有缓存，不能直接把逻辑请求数等同于实测 HBM 字节数，但编程时明确重用能减少对偶然缓存命中的依赖。原页“100 倍成本、约 8 倍速度”和延迟表是示意比较，随工艺、访问粒度和测量方式变化，不能代替目标 GPU 的规格。

误区：**local** 不一定意味着快

CUDA 的 local memory 是线程私有的地址空间概念，并不保证物理上位于寄存器。寄存器不够时 spill 到 local memory，仍可能走设备内存和缓存。反过来，global 地址被 L2 命中时也未必每次访问 DRAM。后面计算内存流量时会明确所讨论的层级。

本节参考（完整书目见讲义末尾）：[slides] · [cuda]

Execution model of a GPU



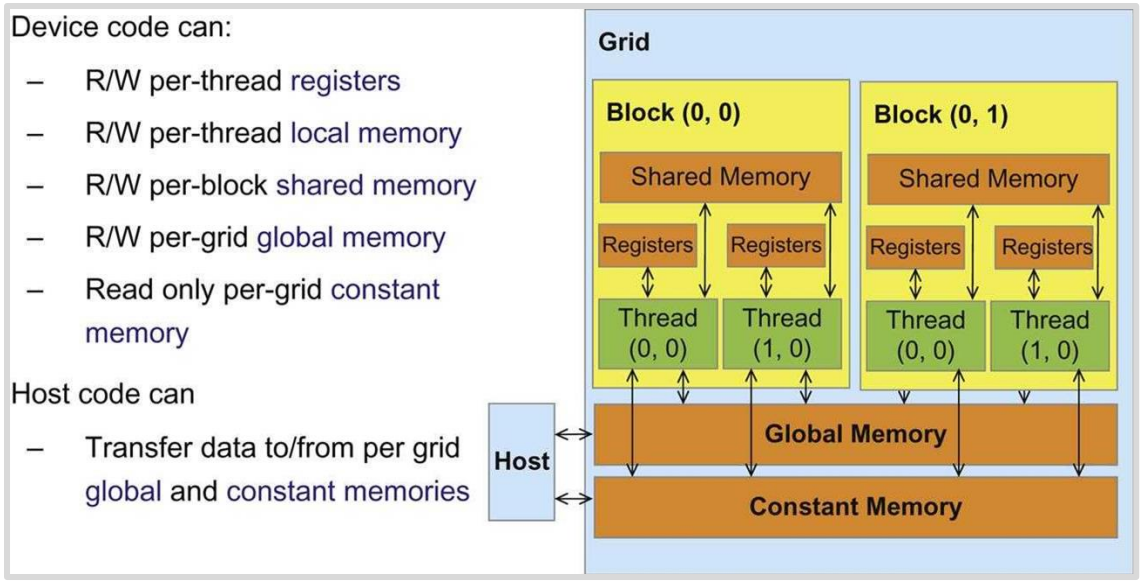
There are 3 important players in the execution model

Threads: Threads 'do the work' in parallel – all threads execute the same instructions but with different inputs (SIMT).

Blocks: Blocks are groups of threads. Each block runs on a SM w/ its own shared memory.

Warp: Threads always execute in a 'warp' of 32 consecutively numbered threads each.

Memory model of a GPU



Each thread can access its own register, and shared memory within the block.

Information that goes across blocks need to be read/written to global memory (slow)

第 5 讲 · 原页 11–12 · 中文讲解

thread、warp、block 与可见性

层级与一个可手算的映射

一次 kernel launch 产生一个 grid，grid 包含许多 block，block 包含线程。通常一个 block 的线程共同驻留在某一个 SM 上，block 不跨 SM 拆开；一个 SM 可以同时承载多个 block。NVIDIA warp 由连续编号的 32 个线程组成。编者例子：处理 1000 个元素，每块 256 个线程，需要 ceil(1000/256)=4 块，每块 8 个 warp，总共启动 1024 个线程；最后 24 个线程通过 i<1000 的边界条件避免越界。块的执行次序不可依赖。

SIMT 的准确读法

“同一指令、不同数据”主要针对 warp 内的执行方式，不是整张 GPU 上所有线程永远在同一条指令。每个线程有自己的索引和寄存器状态；不同 warp 可处于不同进度。warp 内若走不同条件路径，硬件需要用活动掩码等机制处理，不能期待所有分支都免费并行。较新的独立线程调度也不意味着可以省略程序所需的显式同步。

共享数据必须同时考虑位置与同步

第 12 页的颜色表示可访问范围：寄存器通常归线程，shared memory 归 block，global memory 可用于跨 block 的数据交换。一个线程写完 shared memory 后，其他线程在使用它之前通常需要块内屏障，确保所有生产者完成并保证可见性。屏障必须由参与的线程一致到达，不能把整块同步放进只有部分线程执行的分支。普通 block 之间不能依赖块内屏障通信，常见做法是以 kernel 边界分阶段，或采用明确定义的原子/同步机制。现代架构还有 cluster 等扩展，本讲先采用基础模型。

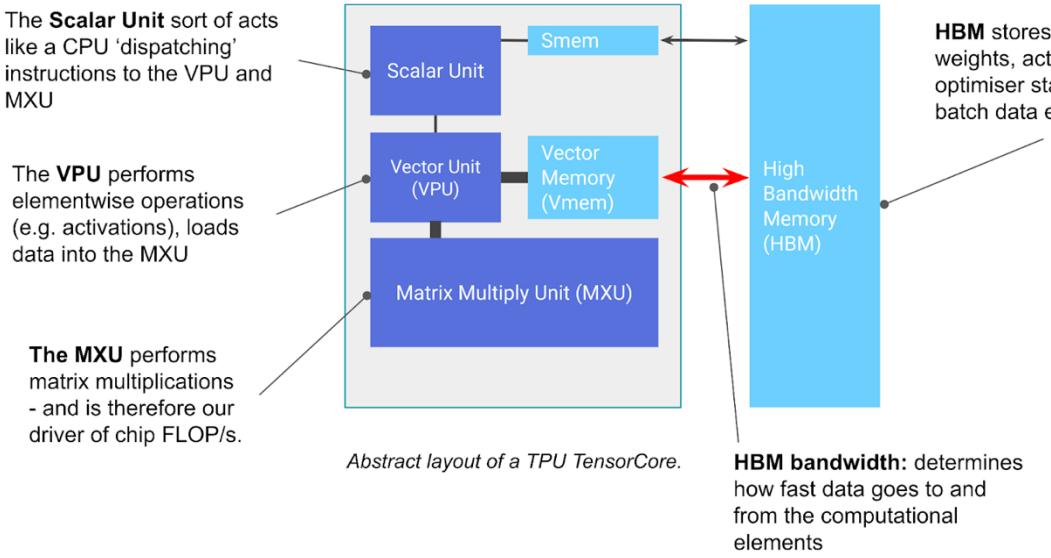
课堂补充 · 00:14:51–00:15:41

字幕中教师在回答提问时区分 block 和 warp：调度器从准备好的 warp 中选择要执行的工作。这里的课堂重点是避免把 block 看成每周期整体锁步执行的单位；本节 1000 元素的数字例子为编者补充。

本节参考（完整书目见讲义末尾）：[slides] · [cuda] · [captions]

Side thread – What about TPUs?

GPUs, TPUs, and many other accelerators are at a high level, similar



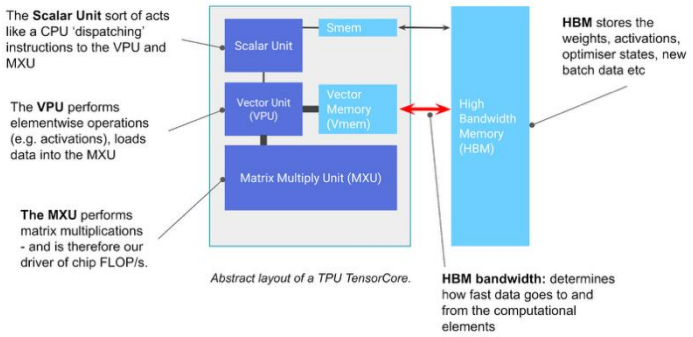
Abstract layout of a TPU TensorCore.

A GPU has *more SMs*
TPUs has fewer TCs
(but similar matmul perf)

Core structure – lightweight control, fast (big) matmul unit, fast memory.

Differences - how the accelerators are networked (in the parallelism lecture)
- no warps (just blocks – tradeoffs in matmul vs non-matmul)

Side thread – What about TPUs?



Abstract layout of a TPU TensorCore.

GPU	TPU	What is it?
Streaming Multiprocessor (SM)	Tensor Core	Core "cell" that contains other units
Warp Scheduler	VPU	SIMD vector arithmetic unit
CUDA Core	VPU ALU	SIMD ALU
SMEM (L1 Cache)	VMEM	Fast on-chip cache memory
Tensor Core	MXU	Matrix multiplication unit
HBM (aka GMEM)	HBM	High bandwidth high capacity memory

GPU	TPU	H100 #	TPU v5p #
SM (streaming multiprocessor)	Tensor Core	132	2
Warp Scheduler	VPU slots	528	8
SMEM (L1 cache)	VMEM	32MB	128MB
Registers	Vector Registers (VRegs)	32MB	256kB
Tensor Core	MXU	528	8

Core structure – lightweight control, fast (big) matmul unit, fast memory.

https://jax-ml.github.io/scaling-book/gpus/

第 5 讲 · 原页 13–14 · 中文讲解

TPU 的相似性与术语差异

把图按功能对应起来

原图中的 TPU TensorCore 包括矩阵运算单元 MXU、向量单元、标量控制及片上内存；GPU 的 SM 也有算术、控制和近端存储。共同原则是用轻量控制驱动大量规则计算，让矩阵单元尽量复用输入。不要因为名称相似就把 TPU TensorCore 与 NVIDIA Tensor Core 当成同一粒度：前者是较大计算核心的名称，后者是 GPU SM 内的专用矩阵运算资源。

编者补充：矩阵阵列为什么适合规则运算

理解 MXU 可以先想象一个乘加网格：A 的元素沿一方向传播，B 沿另一方向传播，在相遇处做乘加并累积局部结果。数据经过多个单元时重复使用，因此减少外围数据传输。矩阵足够大且形状合适时容易填满网格；很小、不规则或包含大量非矩阵算子的任务可能利用率低。GPU 的许多 SM 与 TPU 较少的大核心是原图所比较设备的组织方式，不代表每代芯片都固定如此。

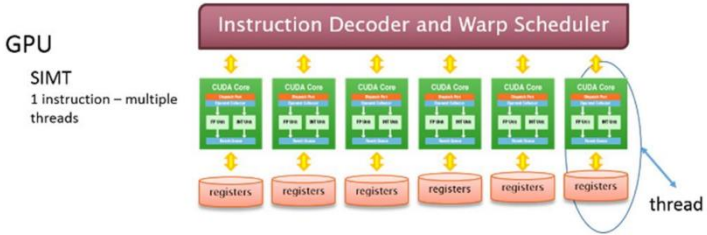
边界与后续联系

第 14 页参数表用于比较特定设备的存储容量、带宽与计算单元，而不是据一个核心数量判断胜负。TPU 不采用 CUDA 的 warp 编程模型，仍需要把张量分块并管理局部存储。本讲只建立功能上的对应；多芯片互联、通信拓扑和分片策略属于后续并行训练内容。

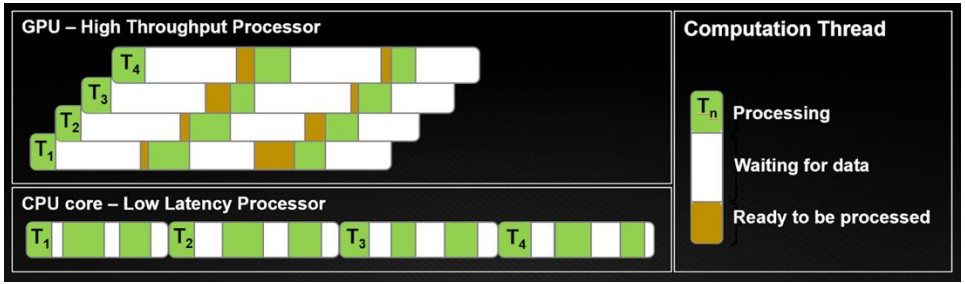
本节参考（完整书目见讲义末尾）：[slides] · [tpu-book]

Strengths of the GPU model

- Easily scales up hard workloads (by adding more SMs)
- Easy (?) to program due to the SIMT model



- Threads are ‘lightweight’ and can be stopped and started



GPUs as fast matrix multipliers

Fast Matrix Multiplies using Graphics Hardware

E. Scott Larsen
Department of Computer Science
University of North Carolina at Chapel Hill
Chapel Hill, NC 27599-3175 USA
larsene@cs.unc.edu

David McAllister
Department of Computer Science
University of North Carolina at Chapel Hill
Chapel Hill, NC 27599-3175 USA
davemc@cs.unc.edu

Implementation

We mention here some observations we made during our implementation that may be of interest to those duplicating our results.

Refresh Rate We found that setting the refresh rate on the monitor as low as possible made marginal improvements (about 10%).

RGBA We found that 4 numbers can be packed into a single pixel, by setting the red, green, blue, and alpha channels to different values.

Texture Format Changing the format of the texture creation and read-back from RGBA to ABGR_EXT (in OpenGL) gave about 40% improvement on our hardware. This is because the hardware driver avoids reformatting the data from the application format to the card format. There is a number of options here, with near equal performance for each option except the one used natively on the specific hardware. The native format should give significant improvement.

Full Screen Running full screen instead of in a window provides improved performance.

Various other optimizations yielded minor (<1%) improvements.

Early days of NVIDIA GPUs – programmable shaders. Researchers hacked this to do matmuls

第 5 讲 · 原页 15–16 · 中文讲解

轻量线程与矩阵乘法的历史联系

为什么线程切换能提高吞吐

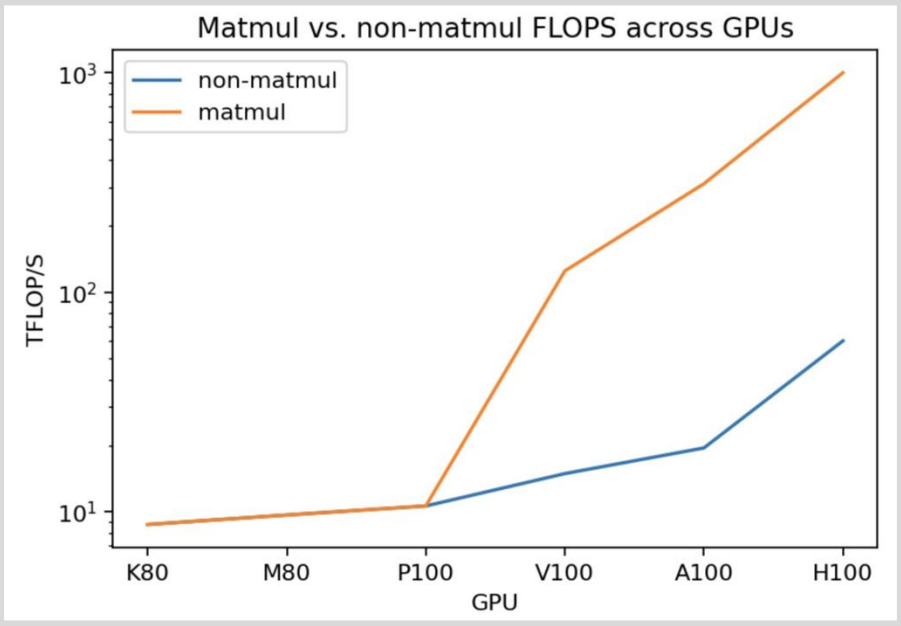
第 15 页下图将 CPU 的较串行时间线与 GPU 的多组线程交错时间线对照。GPU 上许多线程的执行状态已保存在硬件资源中，调度就绪 warp 的成本可以很低，从而穿插执行来隐藏等待。这种轻量是有条件的：所有驻留线程共享有限寄存器、shared memory 和调度槽；启动更多线程不自动增加吞吐。大量任务也必须独立，否则等待依赖仍会造成空闲。

为什么图形硬件适合矩阵乘法

第 16 页早期论文截图展示在可编程 shader 上实现矩阵乘法的历史。图形任务对许多像素/顶点重复相似计算，而矩阵乘法对许多输出元素重复点积，两者都可分解为大量规则工作。对 C=A B 而言，每个 C[i,j] 的结果独立，但 A 的行和 B 的列会被反复使用，所以它不仅容易并行，也有机会通过共享内存重用大幅减少输入流量。历史页的意义是解释工作负载与硬件设计如何相互促进，不要求学习旧图形 API。

本节参考（完整书目见讲义末尾）：[slides]

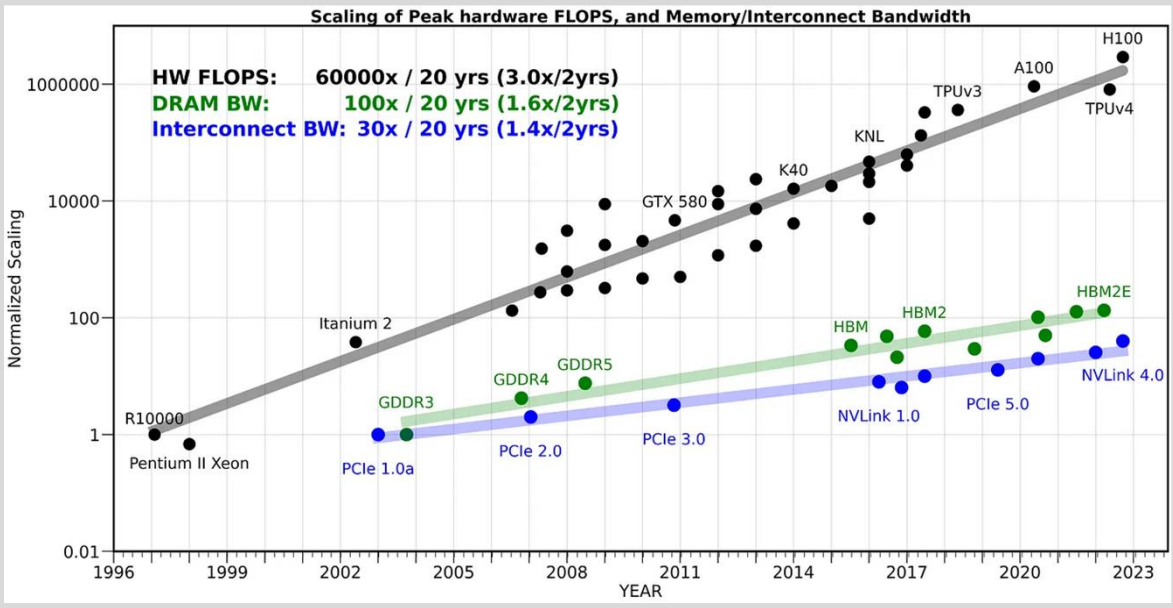
New matmul hardware means matmuls are fast and special



Tensor cores (introduced in V, T series) are specialized matrix multiplication circuits.

Matmuls are >10x faster than other floating point ops!

Compute scaling is faster than memory scaling



https://medium.com/riselab/ai-and-memory-wall-2cb4265cb0b8

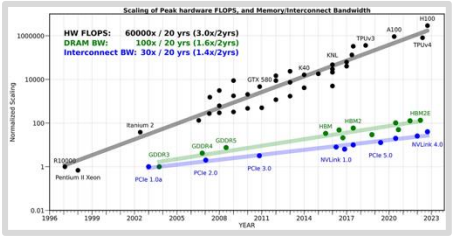
FLOPs scale faster than memory – it's hard to keep our compute units fed with data!

Recap: GPUs – what are they and how do they work

- GPUs are massively parallel – same instructions applied across many workers



- Compute (and esp matmuls) have scaled faster than memory



- We have to respect the memory hierarchy to make things go fast.

TABLE IV
THE MEMORY ACCESSES LATENCIES

Memory type	*CPI (cycles)*
Global memory	290
L2 cache	200
L1 cache	33
Shared Memory (ld/st)	(23/19)

第 5 讲 · 原页 17–19 · 中文讲解

Tensor Core 与计算—带宽失衡

矩阵乘法为什么成为特殊快路径

Tensor Core 用专用数据路径执行小矩阵的乘加，再组合成大 GEMM。第 17 页图中 matmul 曲线比一般非矩阵操作增长更快，表明“一个 FLOP”不是统一成本单位：普通加法、指数函数、不同精度的矩阵乘加可能走不同硬件。比较峰值时须指定数据类型、累加类型、是否使用稀疏性等条件。原稿“超过 10 倍”描述相关硬件与精度的对比，不能据此把任何运算改写成矩阵就断言十倍加速。

用比值理解 **memory wall**

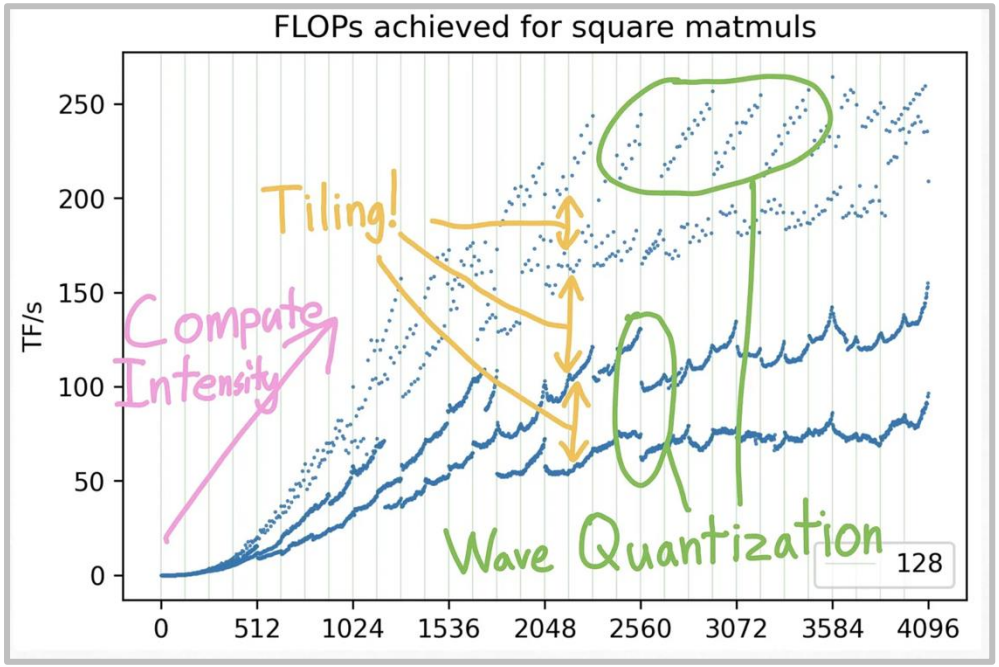
第 18 页把计算能力增长与内存/互联带宽增长放在一起。若峰值 P 增长比带宽 B 快，比例 P/B 的单位是 FLOP/B，表示每搬一字节至少要有多少运算才有机会喂满计算单元。编者例子：P=100 TFLOP/s、B=1 TB/s 时需约 100 FLOP/B；若 P 翻倍而 B 不变，门槛变成 200 FLOP/B。算子不变时，换更强计算硬件可能几乎没有收益。

三条结论连接到下一部分

第 19 页总结的是同一条因果链：大量并行执行单元提供吞吐，矩阵专用硬件让计算更便宜，存储层级决定数据能否及时供应。接下来不是逐项背诵优化技巧，而是每次问“哪一级的哪些字节被省掉了，是否引入新的计算或资源压力”。这也解释 FlashAttention 为什么主要重排数据移动，而不需要改变注意力定义。

本节参考（完整书目见讲义末尾）：[slides]

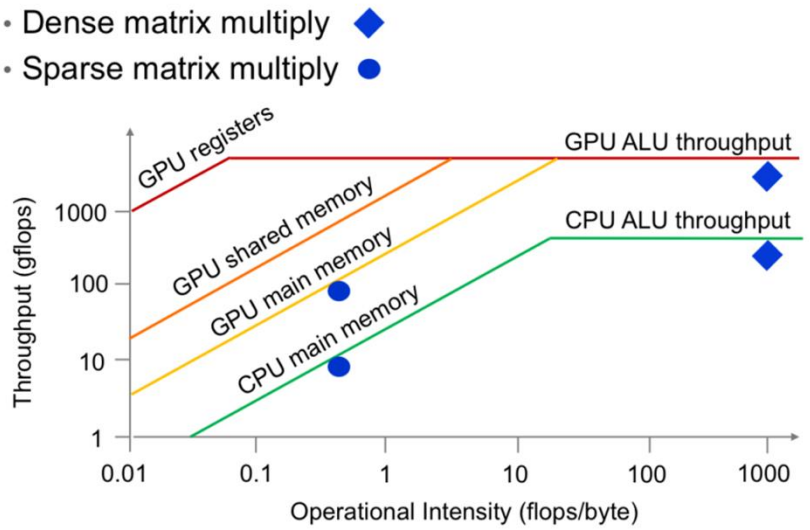
Part 2: Making ML workloads fast on a GPU



Performance on a GPU can be complex, even for something as simple as a square matmul

What makes ML workloads fast?

The roofline model



Key to this section: **how do we avoid being memory bound?**

第 5 讲 · 原页 20–21 · 中文讲解

Roofline：从运算数和字节数推导性能上界

先建立量纲一致的模型

记 F 为浮点运算数，Q 为某级内存传输字节数，P 为同一运算类型的峰值 FLOP/s，B 为该级内存带宽 B/s，I=F/Q 为算术强度 FLOP/B。只考虑计算和传输、允许理想重叠时，时间 t 至少为 max(F/P,Q/B)，所以有效吞吐 F/t ≤ min(P,B I)。这就是 roofline：左边斜线受带宽限制，右边平台受计算限制，转折点 I*=P/B。若两部分不能重叠，实际时间还会更长。

编者手算：同样 **FLOPs** 可以完全不同速

设 $F=10^9$ FLOPs，$P=100\times10^{12}$ FLOP/s，$B=10^{12}$ B/s。若 $Q=10^9$ B，则计算只需下界 10 微秒，而数据搬运需 1 毫秒，明显带宽受限。若靠重用使 $Q=10^7$ B，则两项下界都是 10 微秒。并非实际代码必然达到下界：启动延迟、低占用率、同步、缓存失效和小规模效应都可能让它更慢。

怎样读第 **20** 页锯齿与第 **21** 页屋顶

第 20 页横轴是矩阵规模，通常随着规模增大，重用与并行度变好，所以吞吐上升；但它不是严格的 I 横轴，不能把每个凹陷都解释为算术强度下降。第 21 页标准 roofline 才以 FLOP/B 为横轴，带宽决定斜线斜率，专用矩阵单元与普通 ALU 可对应不同平台。实际曲线远低于屋顶时，要继续检查分块、调度和访存，后面会解释台阶与周期性波动。

课堂补充 · **00:31:11–00:32:21**

教师把曲线的斜段称为 memory-bound、平台称为 compute-bound，强调增加计算单元无法直接救活带宽受限工作。对自学者还需补一条边界：目标是减少总耗时，不是一定把所有算子推到平台；ReLU 这样的工作本身就缺少重用，接近其带宽上限也可以很高效。

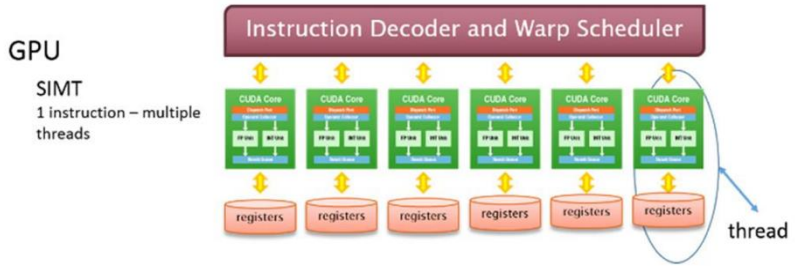
本节参考（完整书目见讲义末尾）：[slides] · [captions]

How do we make GPUs go fast?

1. **Control divergence (not a memory bottleneck..)**
2. **Low precision computation**
3. **Operator fusion**
4. **Recomputation**
5. **Coalescing memory**
6. **Tiling**

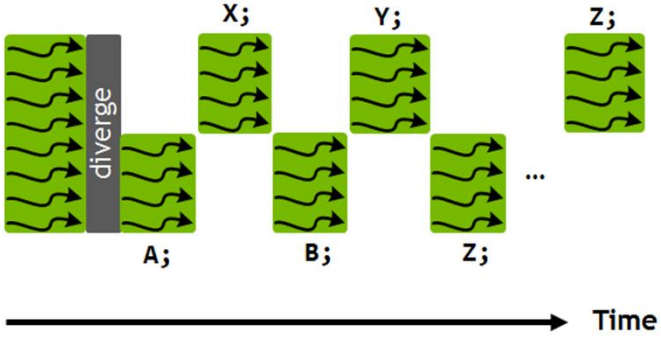
Control divergence (not a memory issue)

GPUs operate in a SIMT model – every thread in a warp is executing the same instruction



Conditionals are fine, but lead to significant overhead from the execution model

```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
```



第 5 讲 · 原页 22–23 · 中文讲解

控制分歧：在内存优化之前看执行路径

同一 **warp** 中不同分支发生了什么

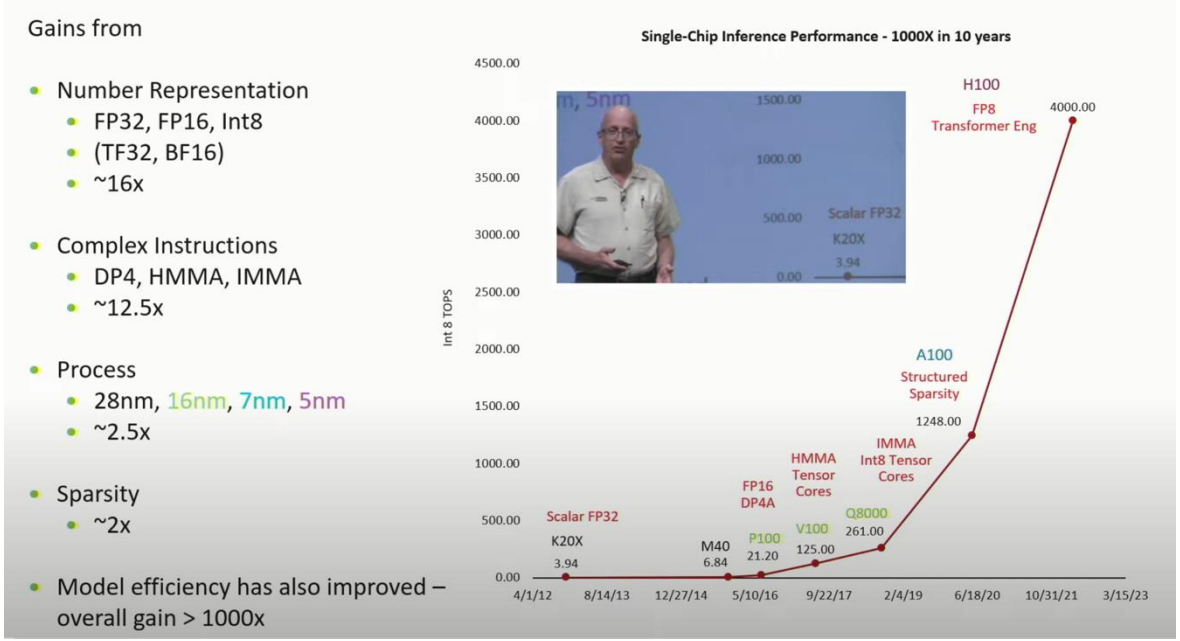
第 23 页代码 if(threadIdx.x<4) 展示部分线程走 A/B，另一些走 X/Y。warp 通常需要执行各条实际被选择的路径，并用掩码让当前路径之外的线程不产生效果，所以总成本可能接近多条路径成本之和，而不是最大值。重要的是同一 warp 内是否分歧：若一个 warp 全走 A，另一个全走 B，二者各自一致，情况与 warp 内一半 A 一半 B 不同。

编者例子与边界

设 32 个线程中前 16 个执行 10 条 A 指令，后 16 个执行 10 条 B 指令。简化模型下需要约 20 条分路径指令，每次只有半数 lane 活跃。若按数据重新分组，让每个 warp 统一走同一路径，可减少这种空槽，但重排数据也有成本。很短的条件可能被编译器改为谓词执行，具体代价还依赖编译结果。不要为消除一个 cheap 分支无条件做大量昂贵计算。第 22 页后续技巧主要对数据流量下手，控制分歧则说明并非所有性能问题都是内存瓶颈。

本节参考（完整书目见讲义末尾）：[slides] · [cuda]

Trick 1: Low precision computation



If you have fewer bits, you have fewer bits to move

Low precision improves arithmetic intensity

Example: elementwise ReLU ($x = \max(0, x)$) on a vector of size n**.**

(Float 32 case)

Memory access: 1 read (x), 1 write (if x < 0), float 32 = 4 bytes per op

Operations: 1 comparison op, 1 FLOP.

Intensity: 8 bytes / FLOP

(Float 16)

Memory access: 1 read (x), 1 write (if x < 0), float 16 = 2 bytes per op

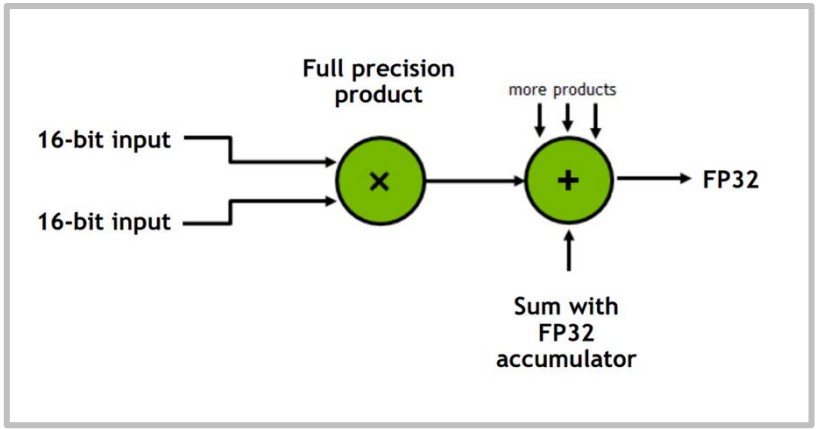
Operations: 1 comparison op, 1 FLOP.

Intensity: 4 bytes / FLOP

Low precision drives faster matrix multiplies

Lots of operations in modern GPUs are accelerated via low / mixed precision operations

Tensor cores



Operations that can use 16-bit storage (FP16/BF16)

- Matrix multiplications
- Most pointwise operations (e.g. relu, tanh, add, sub, mul)

Operations that need more precision (FP32/FP16)

- Adding small values to large sums can lead to rounding errors
- Reduction operations (e.g. sum, softmax, normalization)

Operations that need more range (FP32/BF16)

- Pointwise operations where $|f(x)| \gg |x|$ (e.g. exp, log, pow)
- Loss functions

https://nvlabs.github.io/eccv2020-mixed-precision-tutorial/files/dusan_stosic-training-neural-networks-with-tensor-cores.pdf

第 5 讲 · 原页 24–26 · 中文讲解

低精度：少搬字节与更快乘加是两件事

逐元素 **ReLU** 的完整计数

考虑独立输出 y=max(0,x)，每个元素需读取 x 并写入 y，不论 x 正负都要生成输出。按一次比较/选择记作 1 个 op 的教学口径，FP32 每元素读 4 B、写 4 B，共 8 B，因此 I=1/8 op/B；FP16 每元素共 4 B，因此 I=1/4 op/B。比较不一定按硬件浮点加乘的 FLOP 峰值计量，这里用 op 更严谨。对于 n 个元素，只需把读写量和运算数都乘 n，比值不变。

原页单位与课堂澄清 · **00:35:45–00:36:16**

第 25 页将 8 bytes/FLOP 和 4 bytes/FLOP 标作 Intensity，它们实际上是常规定义算术强度的倒数。字幕 00:36:11–00:36:16 中教师也明确说这是 inverse of arithmetic intensity，应取反。原页括号“if x<0”容易被理解为只写负数，本段采用通常 out-of-place ReLU 的一读一写计数；原地条件存储的流量依赖实现，不能与这个标准算例混算。

混合精度为什么还需要高精度

第 26 页 Tensor Core 示意图区分低精度输入与较高精度累加：C[i,j] 是很多乘积之和，累加误差可能随求和长度增大，所以输入 FP16/BF16、累加 FP32 是常见组合。FP16 有 5 位指数、10 位显式尾数，BF16 有 8 位指数、7 位显式尾数，两者另有 1 位符号；BF16 范围更宽，FP16 对同一量级的表示通常更精细。两者都占 2 B，但指数范围与有效精度不同；“都是 16 位”不意味着训练行为相同。低精度可能同时减少搬运字节并启用更快的矩阵路径，两个收益应分开计算。

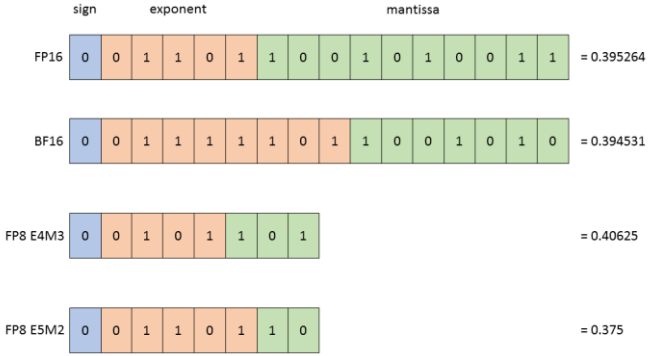
编者补充：为什么理论两倍不是保证

如果原算子完全受 HBM 带宽限制，所有有效传输真的减半且转换开销可忽略，才有约两倍的理想时间改善。若需要额外量化、保存尺度、高精度主权重或执行许多非矩阵运算，端到端收益会小得多。数值上还需检查溢出、下溢、舍入及训练损失；低精度是效率与误差的设计选择，不是无条件压缩开关。

本节参考（完整书目见讲义末尾）：[slides] · [captions]

Frontiers in low precision

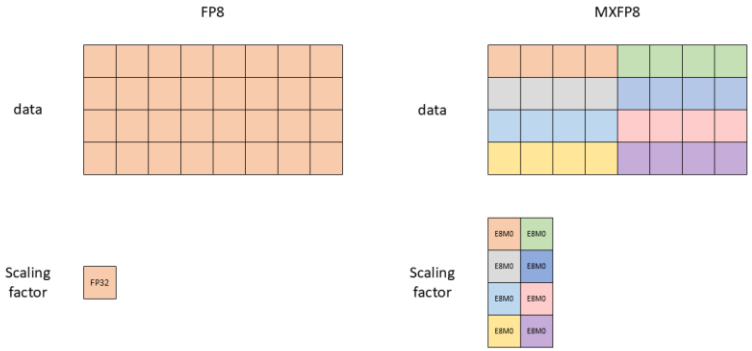
Very low precision (FP8) with different tradeoffs

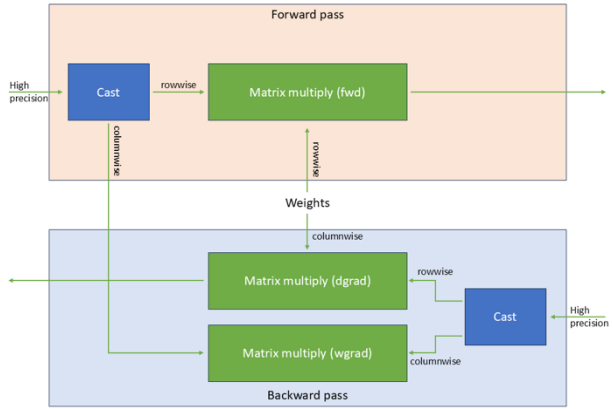


MXFP8 has many *interesting* things about it

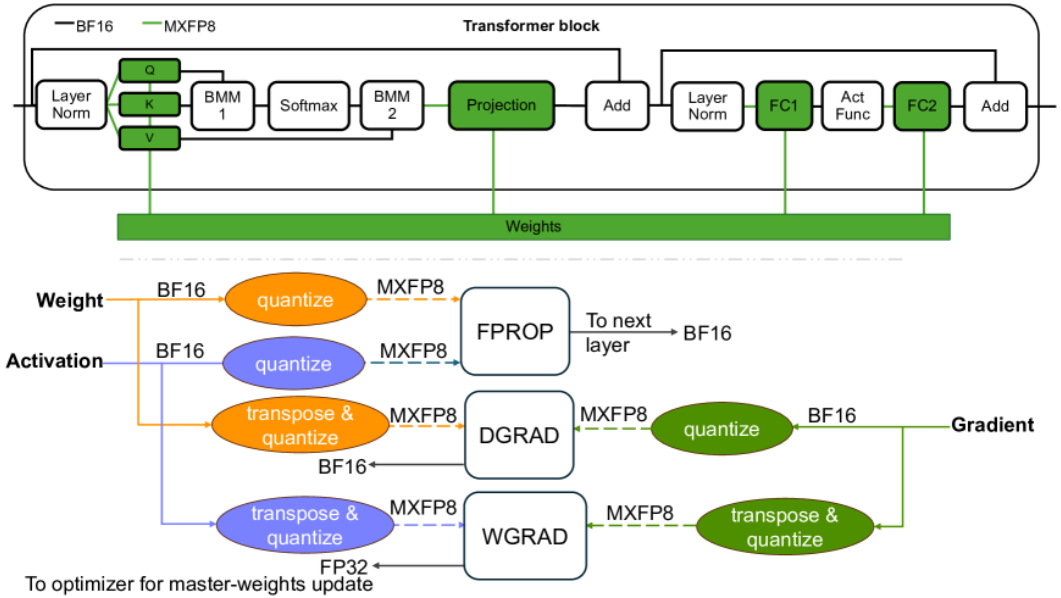
- Uses E4M3 (more mantissa) due to more scale factors
- Scale factors themselves are FP8 (E8M0), 1 per 32
- Transposes are now nontrivial!

Multiple scaling factors MXFP8 (Blackwell)





MXFP8 training in practice



Notice – not all weights in MXFP8, transposes also separately quantized

https://arxiv.org/html/2506.08027v2

第 5 讲 · 原页 27–28 · 中文讲解

MXFP8：局部尺度如何改变矩阵布局

从一个尺度到一组尺度

量化可写作 x≈s·q，其中 q 是低精度可表示值，s 是还原尺度。若整个张量共用 s，一个很大的异常值可能迫使尺度变大，使小值挤在很少的量化档位上；分组尺度让不同局部范围使用不同 s。第 27 页左图对比 FP8 的指数位和尾数位，右图用多色条块表示沿某方向分组。MXFP8 常用 E4M3 元素与每 32 个元素一个 E8M0 尺度；后者是以指数编码的尺度，不是普通的 E4M3 浮点数据。

编者小矩阵：转置为什么不是只换索引

用每 2 个元素一组来缩小演示，矩阵 A=[[1,2],[100,200]]。按行分组时，小量级一组、大量级一组，各用合适尺度。转置 A^T=[[1,100],[2,200]] 后，若硬件要求仍沿每行的连续方向分组，则同一组混合小量级与大量级，原来的分组尺度不能原封不动沿用。数学转置没有误差，但把转置结果重新编码成另一方向的分组量化格式会涉及重分组和舍入。这是布局约束造成的额外成本。

读第 28 页训练流程图

图中的前向、权重梯度、输入梯度需要不同矩阵方向；保存或生成正向与转置两种量化表示，正是为了供应各自的矩阵乘法。高精度状态、尺度和部分敏感层仍存在，不能把图看成全模型每个张量都只剩 8 位。论文所研究的 recipe 是具体训练方案；哪层适合量化、是否值得缓存两份低精度视图，需要同时衡量准确性和内存流量。两种方向应分别从高精度源生成，以避免从已量化结果转置再量化引入额外舍入。

课堂补充 · 00:40:31–00:42:46

教师用转置讲解每 32 元素尺度的约束，强调可能需两份量化方向，并指出量化本身有开销，实际收益不是简单的 2 倍。这里的小矩阵是编者补充，课堂关于速度收益的估计只针对所讨论方案，不作为跨模型承诺。

本节参考（完整书目见讲义末尾）：[slides] · [captions] · [mxfp8] · [te]

Frontiers in low precision

MXFP4..

sign	exponent	mantissa	
0	0	0	0 = 0.0
0	0	0	1 = 0.5
0	0	1	0 = 1.0
0	0	1	1 = 1.5
0	1	0	0 = 2.0
0	1	0	1 = 3.0
0	1	1	0 = 4.0
0	1	1	1 = 6.0

sign	exponent	mantissa	
1	0	0	0 = -0.0
1	0	0	1 = -0.5
1	0	1	0 = -1.0
1	0	1	1 = -1.5
1	1	0	0 = -2.0
1	1	0	1 = -3.0
1	1	1	0 = -4.0
1	1	1	1 = -6.0

This is all the values you can represent! 1 per 16 scaling, E4M3 scaling factors.

第 5 讲 · 原页 29–29 · 中文讲解

FP4 的可表示值，以及 MXFP4/NVFP4 的区别

先读懂表格里的数字

原图用符号位、两位指数和一位尾数列出 E2M1 的有限取值。其数值集合可写作 ±{0,0.5,1,1.5,2,3,4,6}，正负零有不同编码，因而 16 种位模式不等于 16 个不同实数。实际重建值还要乘尺度。编者例子：当尺度为 2 时，可表示的正值包括 1、2、3、4、6、8、12；真实值 5 需要舍入到邻近档位，不能因设置尺度就消除所有量化误差。

原稿名称混用提示

原页标题写 MXFP4，同时写“每 16 个元素一个 E4M3 尺度”；按照 NVIDIA 官方格式对照，这组特征对应 NVFP4，而 MXFP4 通常是每 32 个元素一个 E8M0 尺度。NVFP4 还可包含张量级 FP32 尺度。字幕同样沿用了 MXFP4 的称呼，因此不能用字幕证明二者是同一个格式。原图保持不变，学习时请把数据格式 E2M1、分组大小、尺度类型三个属性分别记录。

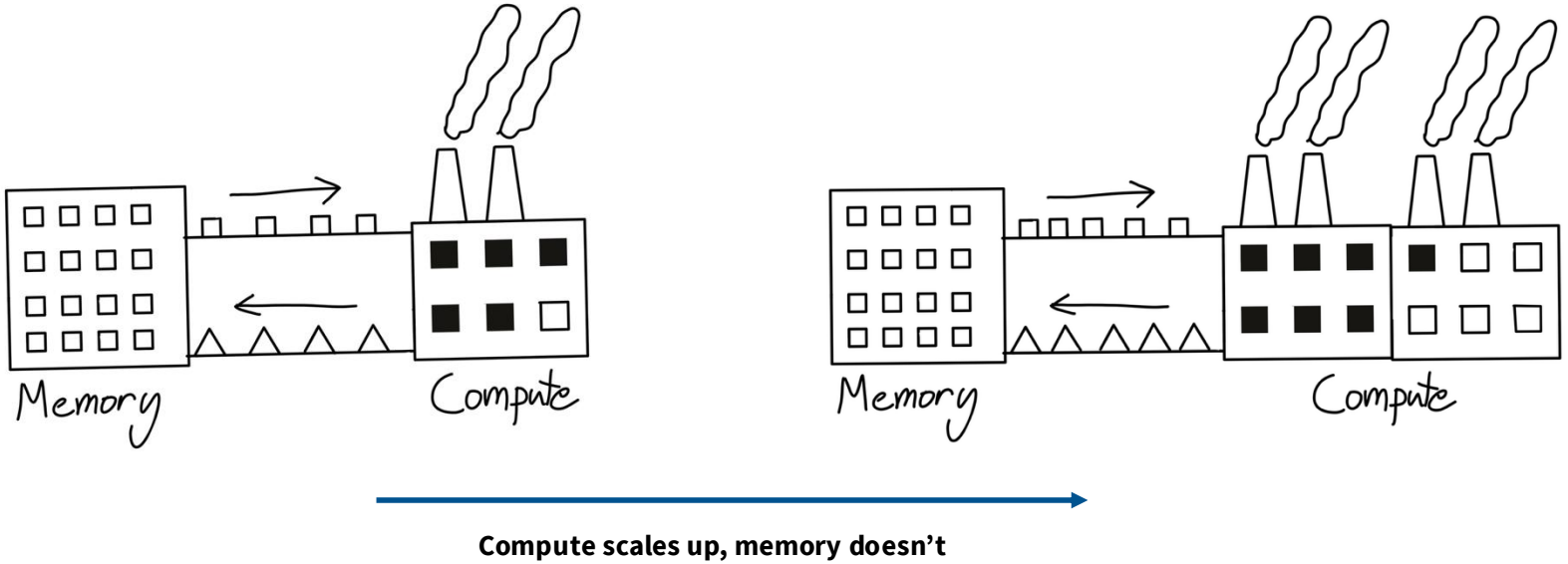
编者补充：元数据与数值边界

若只计分组尺度的摊销，32 个 4-bit 值加一个 8-bit 尺度，占 136 bit，即 4.25 bit/元素；16 个值加一个 8-bit 尺度占 72 bit，即 4.5 bit/元素。实际存储还可能有对齐、张量尺度或额外高精度副本。这也说明标称 FP4 不等于整个训练系统每参数只占 4 bit。更细的尺度可适应局部动态范围，但增加元数据和处理工作；是否更优要看训练稳定性、内核支持与实际吞吐。

本节参考（完整书目见讲义末尾）：[slides] · [nvfp4]

Trick 2: Operator fusion

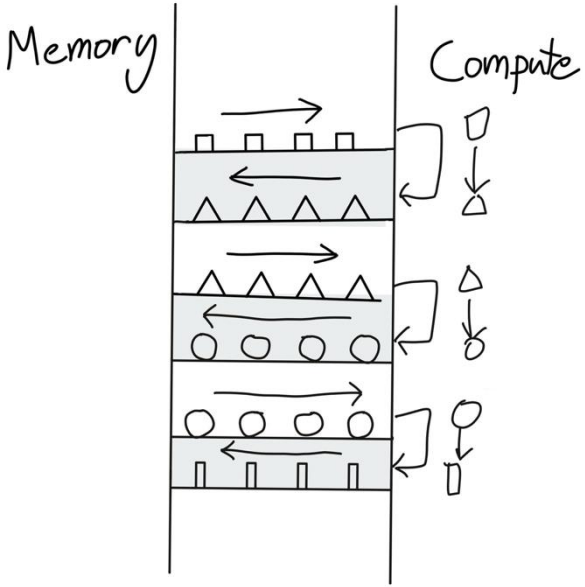
Think of a GPU like a factory – inputs come from a warehouse (memory) and is processed at a factory



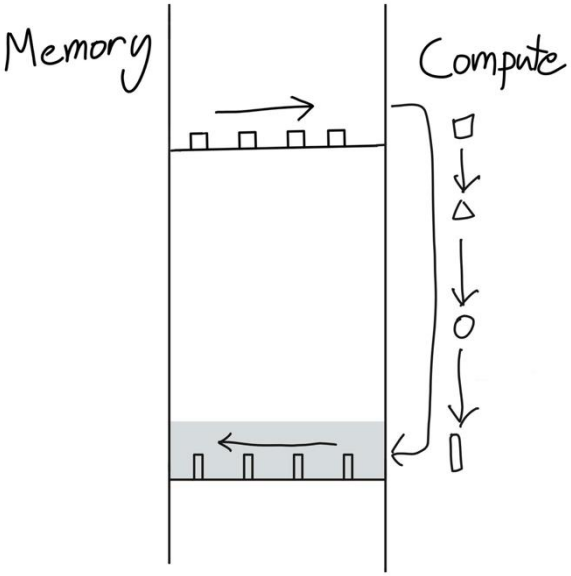
https://horace.io/brrr_intro.html

Operator fusion to minimize memory access

What if we have to do many operations? Shipping back and forth is somewhat silly



Naïve (non-fused)



Fused kernel

第 5 讲 · 原页 30–31 · 中文讲解

算子融合：把中间结果留在工厂里

图中的仓库与工厂对应什么

仓库对应 HBM，工厂对应计算单元及片上临时存储。非融合时，一个算子写出中间张量，下一个算子再读回来；融合后，消费者紧接生产者运行，中间值可留在寄存器或共享内存中，不必绕回 HBM。融合还减少 kernel launch 次数，因此小张量上启动成本的下降也可能重要。两种收益要分别理解。

编者计数：串联三个一元算子

设 $y=h(g(f(x)))$，每个张量有 n 个元素，每元素 b 字节，单独启动三个一元 kernel。若每次均完整读输入、写输出，流量为 $6nb$；融合为一个 kernel 理想上只读 x、写 y，共 $2nb$。数学运算量不变，算术强度约增为 3 倍。这里忽略缓存、地址计算与临时寄存器开销，估算目的在于找到可以消除的中间张量。

融合不是越大越好

若融合后活跃变量很多，寄存器使用可能增大并发生 spill，或减少驻留 block；若跨 block 需要全局同步，也不能随便把两个 kernel 拼起来。矩阵乘法有成熟的专用库实现，强行与复杂旁路融合未必胜过库加一个小 kernel。最值得先找的是逐元素短链、矩阵乘法后的 bias/activation 等消费紧邻生产且重用清楚的地方。

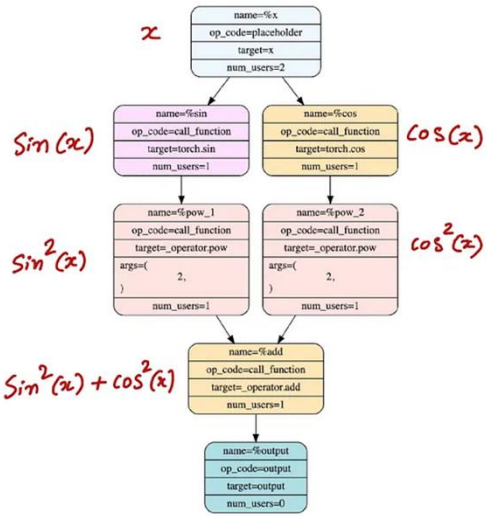
本节参考（完整书目见讲义末尾）：[slides]

Example – sines and cosines

FX GRAPH IR

```
class GraphModule(torch.nn.Module):
    def forward(self, x : torch.Tensor):
        # File: /tmp/ipykernel_2583/1502985755.py:2, code:
        sin = torch.sin(x)
        pow_1 = sin ** 2;  sin = None
        cos = torch.cos(x);  x = None
        pow_2 = cos ** 2;  cos = None
        add = pow_1 + pow_2;  pow_1 = pow_2 = None
        return (add,)
```

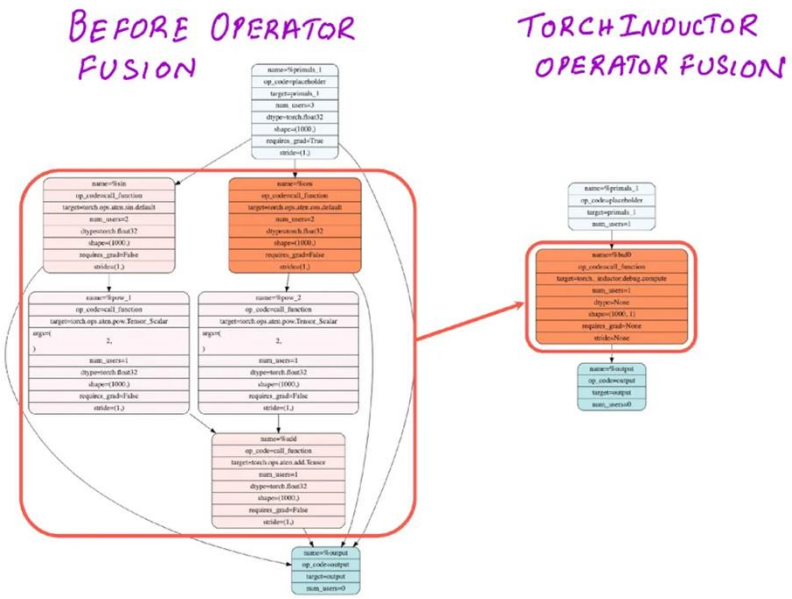
GRAPH VIZ



Computing $\sin^2 x + \cos^2 x$ naively launches 5 CUDA kernels (back and forth)

https://towardsdatascience.com/how-pytorch-2-0-accelerates-deep-learning-with-operator-fusion-and-cpu-gpu-code-generation-35132a85bd26

Fusion example



All 5 pointwise operations can be fused into a single CUDA kernel call.

‘Easy’ fusions like this can be done automatically by compilers (torch.compile)

第 5 讲 · 原页 32–33 · 中文讲解

$\sin^2 x + \cos^2 x$：从计算图数内存往返

逐节点追踪五个 **kernel**

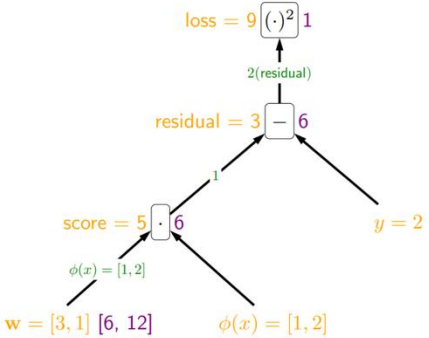
原页的五个逐元素步骤是 $a=\sin(x)$、$b=a^2$、$c=\cos(x)$、$d=c^2$、$y=b+d$。用 bpe 表示每元素字节数，前四个一元 kernel 各需一读一写，第五个加法需两读一写，所以未融合的逻辑流量共 $(2+2+2+2+3)\text{nbpe}=11\text{nbpe}$。融合时可以只读 x 一次，在寄存器中完成中间步骤并写 y，一共 2nbpe。这个 11/2 的比值只是数据流量估算，不是实测加速倍数，因为 sin/cos 运算本身也可能昂贵。

编译器在这里做什么

第 33 页从计算图到单个 kernel 的箭头意味着编译器识别共同输入、安排逐元素操作顺序并生成代码。torch.compile 是可实现此类优化的工具，但实际能否融合取决于后端、图分段、形状和算子支持，不能仅因调用它就断言生成一个 kernel。数学上 $\sin^2 x+\cos^2 x=1$，编译器也未必能在保留浮点舍入、NaN/Inf 语义的前提下把整个表达式换成常数；本例学习的是融合的数据流，而不是依赖恒等式跳过计算。

本节参考（完整书目见讲义末尾）：[slides]

Trick 3: recomputation



$$\text{Loss}(x, y, \mathbf{w}) = (\mathbf{w} \cdot \phi(x) - y)^2$$

$\mathbf{w} = [3, 1], \phi(x) = [1, 2], y = 2$

backpropagation

$\nabla_{\mathbf{w}} \text{Loss}(x, y, \mathbf{w}) = [6, 12]$

Definition: Forward/backward values

Forward: f_i is value for subexpression rooted at i

Backward: $g_i = \frac{\partial \text{loss}}{\partial f_i}$ is how f_i influences loss

Algorithm: backpropagation algorithm

Forward pass: compute each f_i (from leaves to root)

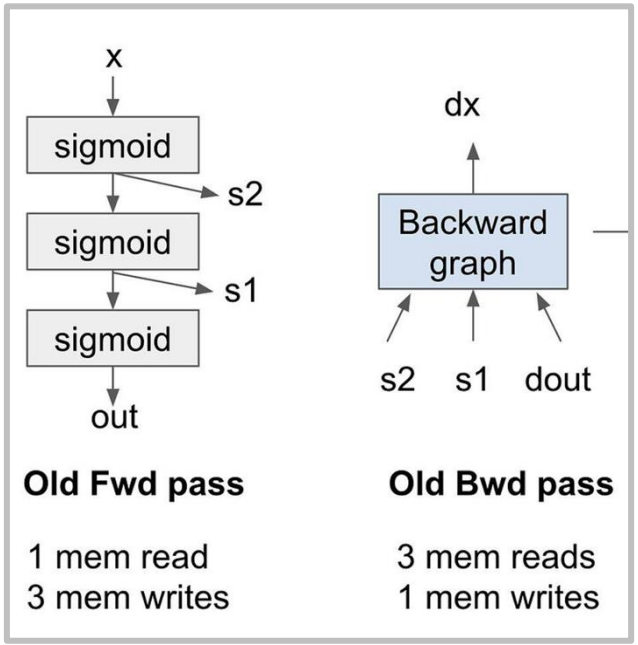
Backward pass: compute each g_i (from root to leaves)

[From cs221]

In backpropagation, we store the activations (yellow) and compute Jacobians (green)

Storing (and retrieving) activations can be expensive!

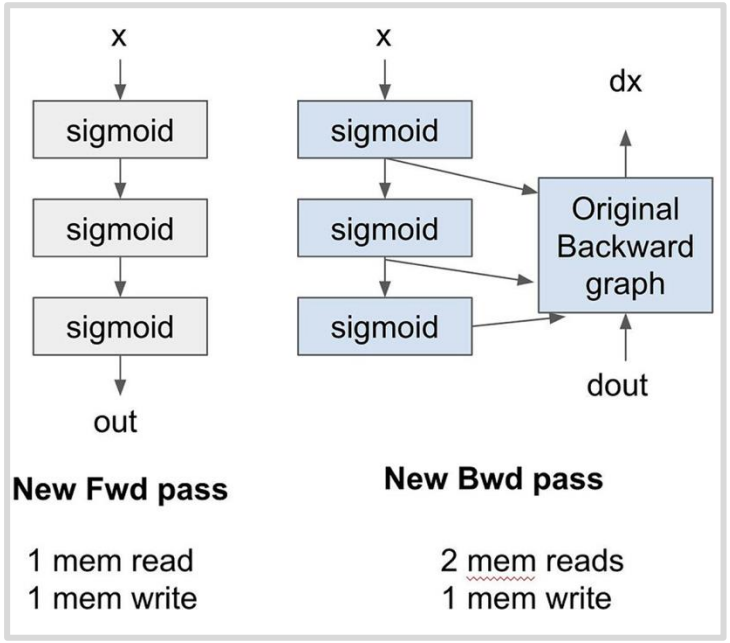
Let’s say we stack 3 sigmoids on top of each other.



This is really terrible for perf – 8 mem read/writes, very low arithmetic intensity.

https://dev-discuss.pytorch.org/t/min-cut-optimal-recomputation-i-e-activation-checkpointing-with-aotautograd/467

Throw away the activations, re-compute them!



Throwing away computation can actually be optimal, w/ 5/8th the memory accesses!

https://dev-discuss.pytorch.org/t/min-cut-optimal-recomputation-i-e-activation-checkpointing-with-aotautograd/467

第 5 讲 · 原页 34–36 · 中文讲解

重计算：为什么多做运算反而能更快

把链式法则与保存激活连起来

设 a=σ(x)、b=σ(a)、y=σ(b)，其中 σ(z)=1/(1+exp(-z))，上游梯度为 g=∂L/∂y。因为 σ′(z)=σ(z)(1−σ(z))，所以 ∂L/∂x = g·y(1−y)·b(1−b)·a(1−a)。反向传播需要的是这些局部值和向量—雅可比乘积，不必真的构造大雅可比矩阵。保存 a、b、y 可避免重算，却增加显存占用和读写。

先解释原页 **8** 次与 **5** 次的口径

本段计数假设前向三个 sigmoid 已融合到一个 kernel，反向各操作也融合；它比较的是是否把激活保存到 HBM，而不是是否融合。若前向三个算子各自独立启动，额外中间读取会改变总数。第 35 页示意图把原前向标作 1 read + 3 writes，原反向标作 3 reads + 1 write，合计 8 次；第 36 页重计算前向是读 x、写 y 两次，反向读 x 和上游梯度、写 dx 三次，合计 5 次。这是示意图所采用的边界和保留值计数，并不是一切三层 sigmoid 实现统一的精确流量。若按完整独立前后向接口，反向还需要外部输入 g，并读取保存的 a、b、y，则常规方案是 4+5=9 次；重计算方案仍可为 2+3=5 次。避免把图示 5/8 与另一种计数边界混算。

编者补充：什么时候值得重算

重新读取 x 后，在反向 kernel 内重建 a、b、y，立即用于导数并丢弃，中间值只在寄存器中短暂存在。设多出的计算为 ΔF、少搬的字节为 ΔQ；在非常简化且有闲置算力的情形，若 ΔF/P 明显小于 ΔQ/B，就可能既省显存又省时间。实际应考虑额外计算是否与传输重叠以及寄存器压力。昂贵矩阵乘法的重算可能增加总耗时，因此 activation checkpointing 常首先是容量换计算，并非总是加速技巧。

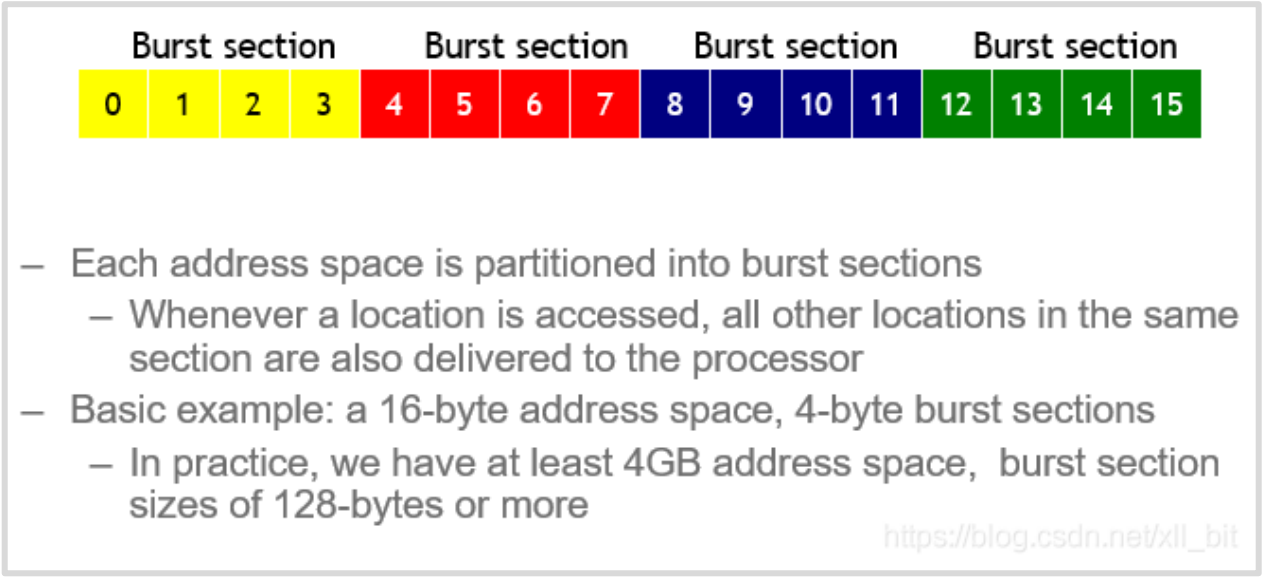
正确性条件与课堂补充 · **00:50:13–00:52:50**

教师用同一 sigmoid 链说明把中间激活丢掉，在反向需要时重新生成。数学上必须生成与原前向一致的值；若包含 dropout，需要保存或重现随机状态，若有可变外部状态或非确定性操作，也需相应处理。FlashAttention 后面会用同样原则重新生成概率块，避免保存整个 N×N 激活矩阵。

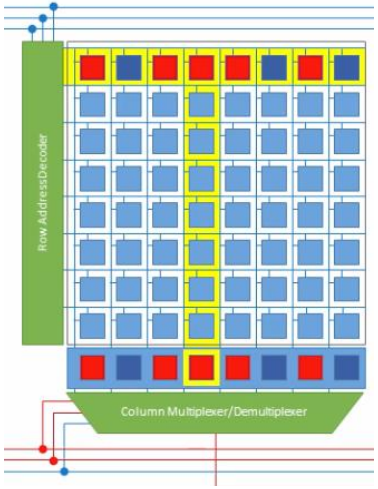
本节参考（完整书目见讲义末尾）：[slides] · [captions] · [recompute]

Trick (?) 4: Memory coalescing and DRAM

DRAM (global memory) is read in ‘burst mode’ – each read gives you many bytes!



[https://blog.csdn.net/xll_bit/article/details/117702476]

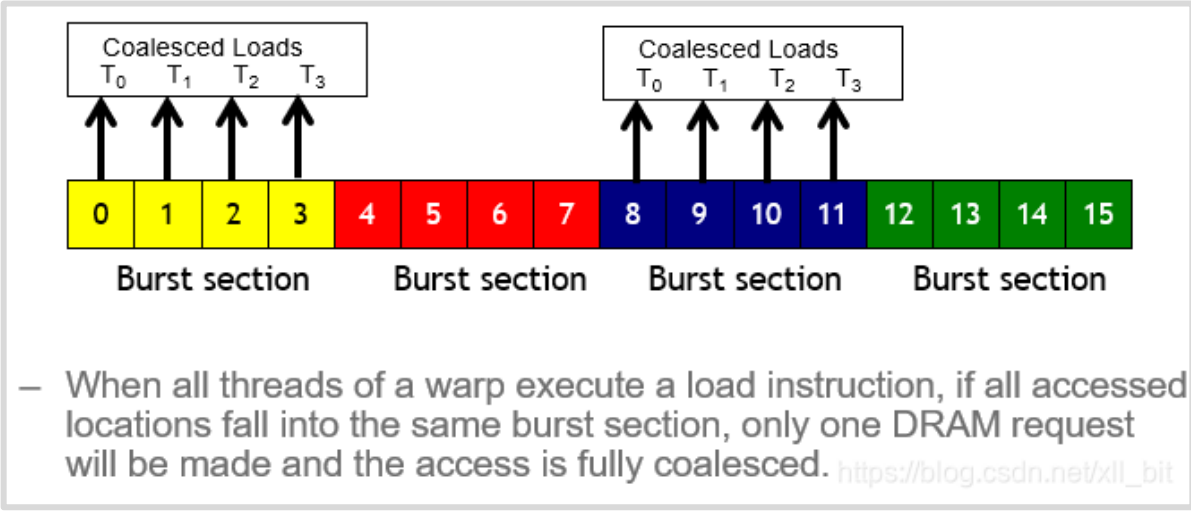


← Burst mode comes from the slow per-row copy to the sense amplifier

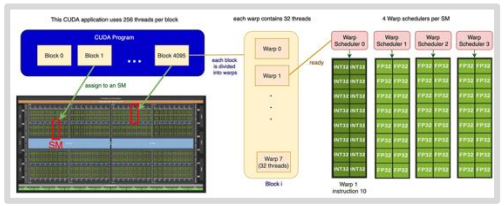
[https://www.youtube.com/watch?v=9BjVUmaXaCQ]

Memory coalescing

Memory accesses are *coalesced* if all the threads (in a warp) fall within the same burst



– When all threads of a warp execute a load instruction, if all accessed locations fall into the same burst section, only one DRAM request will be made and the access is fully coalesced. https://blog.csdn.net/xll_bit



Reminder: a warp is a set of 32 consecutively numbered threads that execute together in a block. Memory accesses happen together

第 5 讲 · 原页 37–38 · 中文讲解

合并访存：按同一条指令的地址集合分析

从 **DRAM** 示意到程序中的一次 **load**

第 37 页 burst 图说明外存不是按每个线程单独请求的几个字节逐个搬运；第 38 页把 warp 内多个线程的地址映射到传输片段。coalescing 的核心是：同一 warp 在同一条访存指令中，请求地址是否集中在少数对齐的内存片段内。相邻线程读相邻元素通常很好。原稿用“同一个 burst”作直觉模型，实际事务粒度、缓存行和 DRAM burst 不能完全混为一谈。

编者手算：为什么连续读更划算

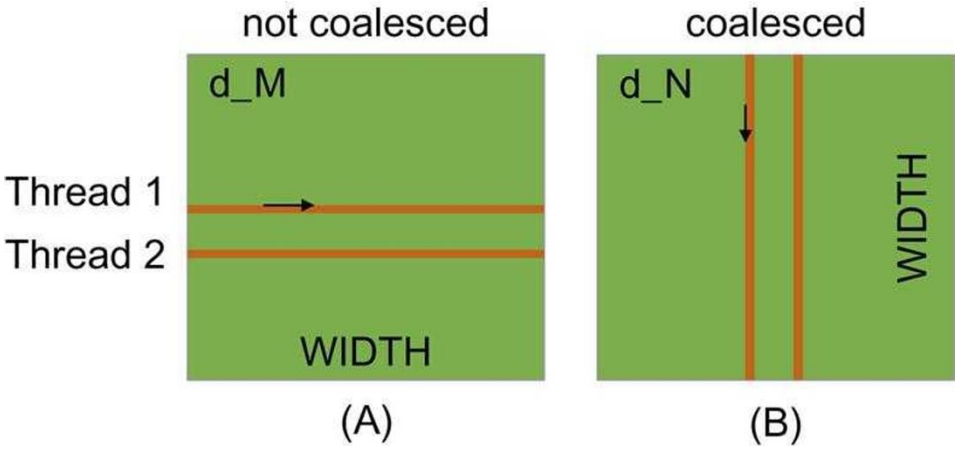
以现代 CUDA 常见的 32 B sector 模型举例，32 个线程各读一个对齐连续 FP32，共需 128 B 有效数据，可覆盖 4 个 sector。若每个线程相隔 32 个 FP32，地址间隔 128 B，可能涉及 32 个不同 sector，共处理 1024 B sector 数据，而有效数据仍为 128 B。这里只比较请求层级的事务覆盖，实际 HBM 流量受缓存命中等因素影响。连续但起点偏移一个 FP32 时，也可能从 4 个 sector 变成 5 个。

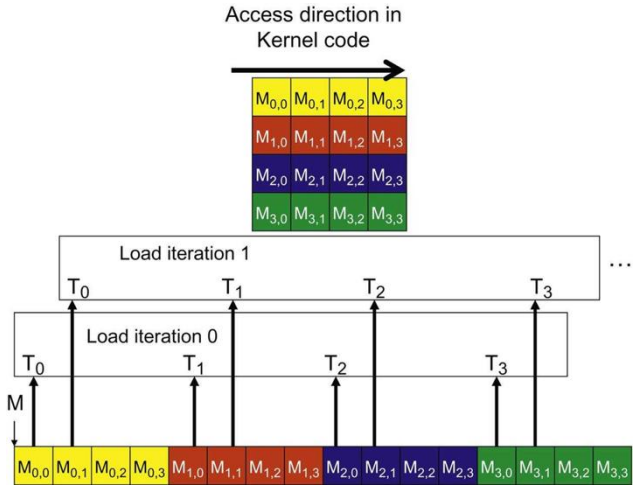
常见误判

不要看单个线程跨很多循环最终是否读遍整行，而要看同一时刻相邻 lane 的地址。广播同一地址也可能高效，并不要求每个线程都必须不同地址。合并访存解决一次请求是否浪费事务，tiling 解决同一值是否反复从远处读取；二者相互补充，不能用其中一个替代另一个。共享内存还有 bank conflict 这类不同机制的问题，不能把一切地址冲突都称为 global memory 不合并。

本节参考（完整书目见讲义末尾）：[slides] · [cuda-best]

Coalescing for matrix multiplication





For row-major matrices – **threads that move along rows are not coalesced**

Note how the second diagram reads the entire vector at each step!

第 5 讲 · 原页 39–39 · 中文讲解

行主序矩阵：分清沿行移动与跨行移动

用地址公式消除语言歧义

对于 row-major 的矩阵 A，元素地址为 base + (i·W+j)·b，其中 W 是行宽、b 是元素字节数。若同一 warp 的 lane t 读取 A[i,j+t]，列索引变化、行索引固定，地址连续；若读取 A[i+t,j]，行索引变化、列索引固定，地址间隔 Wb。原页“threads that move along rows are not coalesced”应结合左图理解为不同线程分布到不同的行，不是说在同一行内连续移动不合并。

把规则代入矩阵乘法

$C[i,j]=\Sigma_k A[i,k]B[k,j]$。若相邻线程负责同一输出行的连续 j，那么固定 k 时，各线程读取同一个 A[i,k] 和连续的 B[k,j]；前者可广播/重用，后者可合并。若相邻线程反而对应连续 i，在同一个 k 下就会跨行读取 A。第 39 页两张绿色矩阵示意图要按这个“固定某次 k，比较多个线程”方式阅读，不能只沿单条箭头追踪一个线程。

编者数字例子

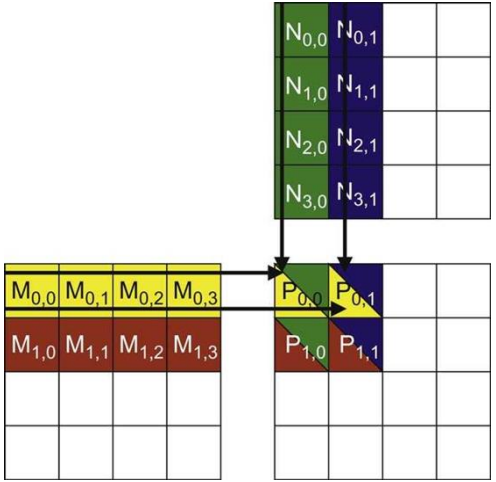
W=1024、FP32 时，A[i,j] 与 A[i,j+1] 相差 4 B，而 A[i,j] 与 A[i+1,j] 相差 4096 B。二者数学上只是相邻元素，但物理存储距离不同。转置视图未必移动实际字节，因此要看 stride；通过合作加载到 shared memory 后改变访问组织，是处理这类布局差异的重要方法。

本节参考（完整书目见讲义末尾）：[slides] · [cuda-best]

Trick 5 (the big one): tiling

Tiling is the idea of grouping and ordering threads to minimize global memory access.

Let’s go back to matrix multiplication..



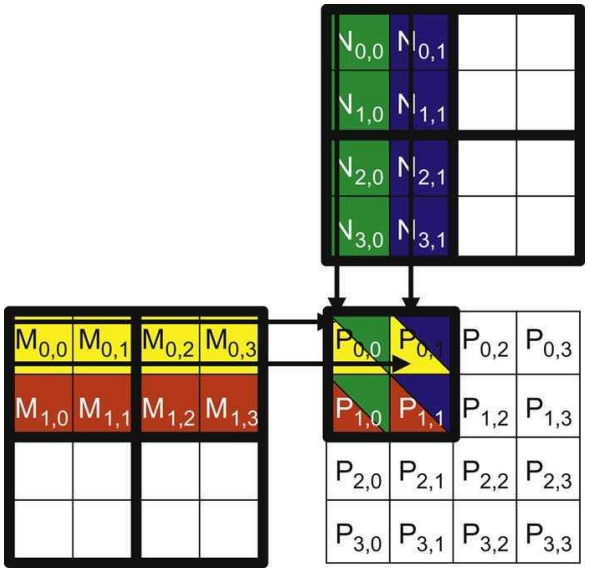
Access order

$thread_{0,0}$	$M_{0,0} * N_{0,0}$	$M_{0,1} * N_{1,0}$	$M_{0,2} * N_{2,0}$	$M_{0,3} * N_{3,0}$
$thread_{0,1}$	$M_{0,0} * N_{0,1}$	$M_{0,1} * N_{1,1}$	$M_{0,2} * N_{2,1}$	$M_{0,3} * N_{3,1}$
$thread_{1,0}$	$M_{1,0} * N_{0,0}$	$M_{1,1} * N_{1,0}$	$M_{1,2} * N_{2,0}$	$M_{1,3} * N_{3,0}$
$thread_{1,1}$	$M_{1,0} * N_{0,1}$	$M_{1,1} * N_{1,1}$	$M_{1,2} * N_{2,1}$	$M_{1,3} * N_{3,1}$

Note that memory access is not coalesced, and repeated (M0,0 and N1,0)

Tiling – store and reuse information in shared memory

Cut up the matrix into smaller ‘tiles’, and load this into shared memory



Compute the matrix multiply in ‘phases’

1. Load $M_{0,0}$ and $N_{0,0}$ tiles into SHM
2. Compute partial sums for P

   (Done with one tile)
3. Load the $M_{0,0}$ and $N_{2,0}$ tile into SHM
4. …

Advantages: repeated reads now access shared, not global memory and memory access can be coalesced

第 5 讲 · 原页 40–41 · 中文讲解

分块矩阵乘法：先搬一小块，再反复使用

两个输入块怎样产生一个输出块

设 C=A B，选定 C 中 T×T 的输出 tile。将归约维 k 按 T 分段，每一阶段把 A 的 T×T 子块和 B 的 T×T 子块合作装入共享内存，再计算对 C tile 的部分贡献。输出累加器在不同阶段间保留，所有 k 段处理完才写回。原图不同颜色对应局部输入块和输出区域；它表达的是沿 k 推进，而不是每次重新计算整张 C。原页第 41 页的 M/N 下标应以这个一般分块关系理解。

编者手算：**4×4** 矩阵、**2×2 tile**

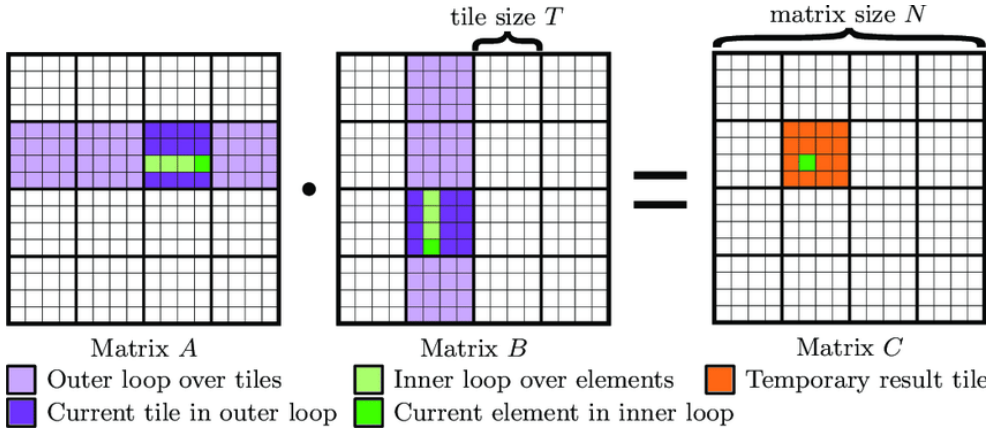
对 C 的左上 2×2 块，第 1 阶段载入 A[0:2,0:2] 与 B[0:2,0:2]，第 2 阶段载入 A[0:2,2:4] 与 B[2:4,0:2]。例如 C[0,0]=A00B00+A01B10+A02B20+A03B30，前两项来自第一阶段，后两项来自第二阶段。每阶段输入共 8 个元素，却能支持 4 个输出各 2 次乘加；每个输入被两个输出重用。合作加载可以按连续地址组织，即使计算阶段各线程需要不同访问方向。

实现顺序中的正确性

一个阶段至少包含加载、块内同步、计算、再次同步后复用 shared buffer。第一次防止有人读到未装好的数据，第二次防止快线程覆盖慢线程尚在使用的数据。高性能内核可双缓冲并重叠加载和计算，但仍需维持等价依赖。tile 是数据和计算分组，block 是线程分组，二者常关联却不必一元素对应一线程；每线程也可以算多个输出。

本节参考（完整书目见讲义末尾）：[slides] · [cuda-best]

Tiling math



Non-tiled matrix multiply: each input is read N times from global memory

Tiled matrix multiply: each input is read $\frac{N}{T}$ times from global memory, and T times within each tile. This is a factor of T reduction in global memory access

第 5 讲 · 原页 42–42 · 中文讲解

分块的流量收益：推导 N/T 从何而来

编者逐步推导

考虑 N×N 矩阵相乘，假设 N 能被 T 整除，先忽略缓存与 C 的读取。朴素方法让每个输出独立做 N 次乘加，需要读 2N 个输入，总输入读数 $2N^3$。分块有 $(N/T)^2$ 个输出 tile，每个 tile 有 N/T 个阶段，每阶段读 $2T^2$ 个输入，所以总输入读数是 $(N/T)^2 \cdot (N/T) \cdot 2T^2 = 2N^3/T$。于是同样的 A 或 B 元素从远端被读取 N/T 次，而每次载入后在片上复用 T 次；相对朴素逻辑输入流量减少 T 倍。

把输出写入和量纲加回来

设每元素 b 字节，C 从零开始累加、最后写回 N^2 个元素，则 $Q \approx b(2N^3/T+N^2)$，乘加按 2 FLOPs 算，$F \approx 2N^3$。因此 $I \approx 2N^3/[b(2N^3/T+N^2)]$；N 远大于 T 时约为 T/b FLOP/B。这里是特定理想分块模型，不是所有 GEMM 的精确 HBM 计数，L2 重用可能进一步降低真实流量，读旧 C 或边界块则增加成本。

编者数字例子与资源限制

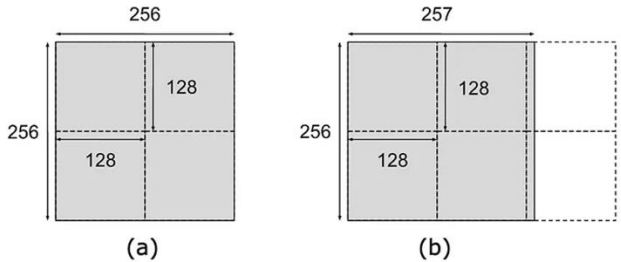
N=1024、T=32、FP32 时，忽略输出的朴素输入量为 8 GiB，分块后为 256 MiB，再加 4 MiB 的输出写入；强度约 7.88 FLOP/B，接近 T/b=8。一个阶段仅 A/B 共享输入就占 $2T^2b$=8 KiB。若 T 变成 64，共享输入增到 32 KiB，输出累加器还会增加。重用随 T 提高，资源占用却按面积增长，因此不能无限放大 tile。

本节参考（完整书目见讲义末尾）：[slides]

Complexities with tiling

Tile sizes may not divide the matrix size and lead to low utilization

Figure 6. Example of tiling with 128x128 thread block tiles. (a) Best case - matrix dimensions are divisible by tile dimensions (b) Worse case - tile quantization results in six thread blocks being launched, two of which waste most of their work.



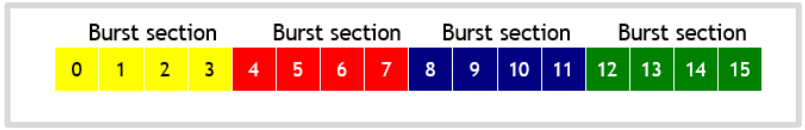
Factors affecting tile sizes

- Coalesced memory access
- Shared memory size
- Divisibility of the matrix dim

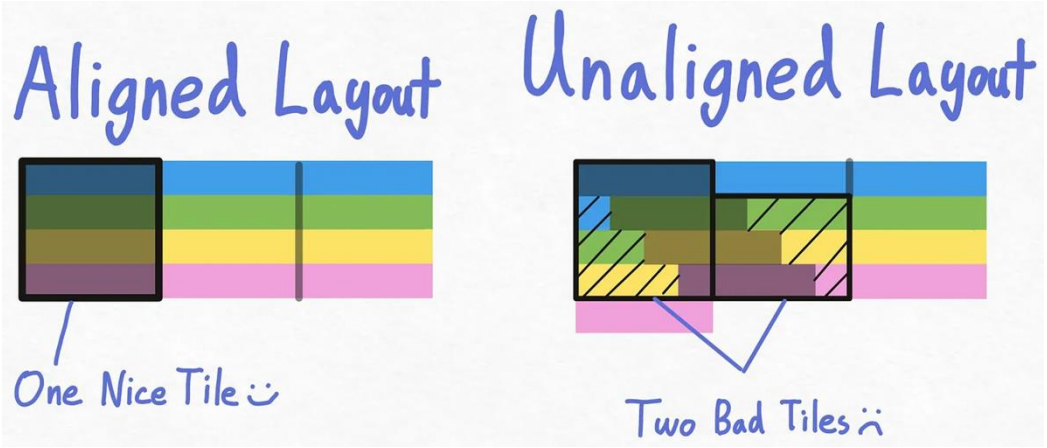
https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html#tile-quant

Complexities with tiling 2 – memory alignment

Memory comes in bursts



Loading tiles are fast if bursts align with the matrix



https://www.thonking.ai/p/what-shapes-do-matrix-multiplications

Coalesced accesses may be impossible depending on the dimension of the matrix..
(have to do padding)

第 5 讲 · 原页 43–44 · 中文讲解

分块的边界成本与内存对齐

第 **43** 页：多出来一行也可能多开很多块

输出 tile 通常有固定形状，矩阵边缘不足一块时需要 mask，硬件/内核仍要承担部分无效工作。编者例子：256×256 输出使用 128×128 tile，只需 4 块；257×257 需要 ceil(257/128)²=9 块，按完整 tile 槽位计的有效比例是 $257^2/(9\cdot128^2)\approx44.8\%$。这不表示每条无效指令都实际执行，也不表示耗时一定翻倍，但解释为什么小幅增加尺寸可能导致吞吐骤降。

第 **44** 页：连续还不够，要看起点和行距

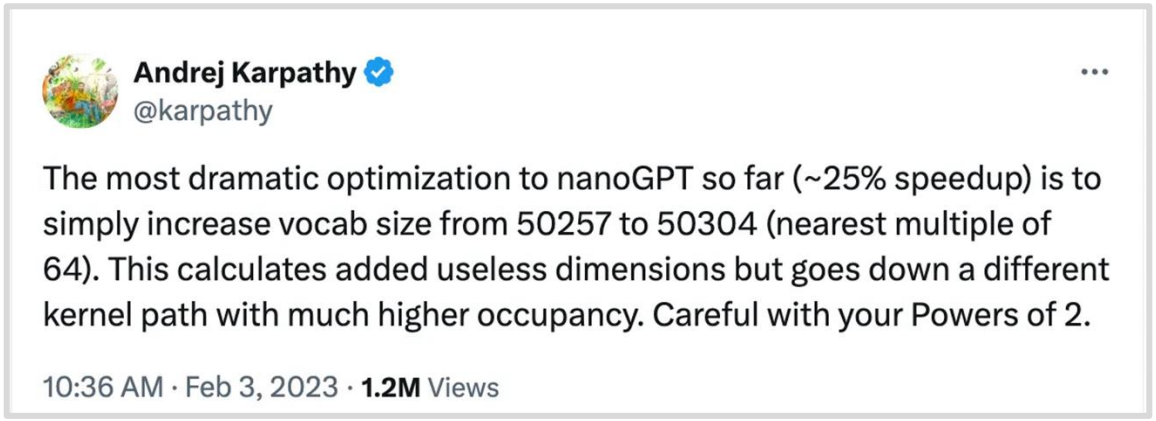
原图左侧 tile 与传输窗口边界对齐，右侧错位会横跨更多窗口。矩阵每行起点由 leading dimension 或 stride 决定；如果行宽字节数不满足合适对齐，后续行的起点会逐行偏移。padding 可让每行起点或矩阵维度更适合硬件，但会多占内存、增加无效计算。原页“可能无法合并”是简化措辞：不对齐通常导致更多事务或效率下降，不是所有此类矩阵都绝对不能进行 coalescing。

选择 **tile** 是多约束问题

大 tile 增加输入重用，小 tile 减少边界浪费并提供更多可分配任务。还需考虑 Tensor Core 的操作粒度、共享内存容量、寄存器需求和驻留 block 数。不存在只看 N 就能决定的通用最优 T。性能调优先用这些约束缩小候选，再在相同 dtype、布局、形状和计时方法下比较实测。

本节参考（完整书目见讲义末尾）：[slides] · [gemm]

Putting it together: understanding a matrix mystery



Andrej Karpathy ✓
@karpathy

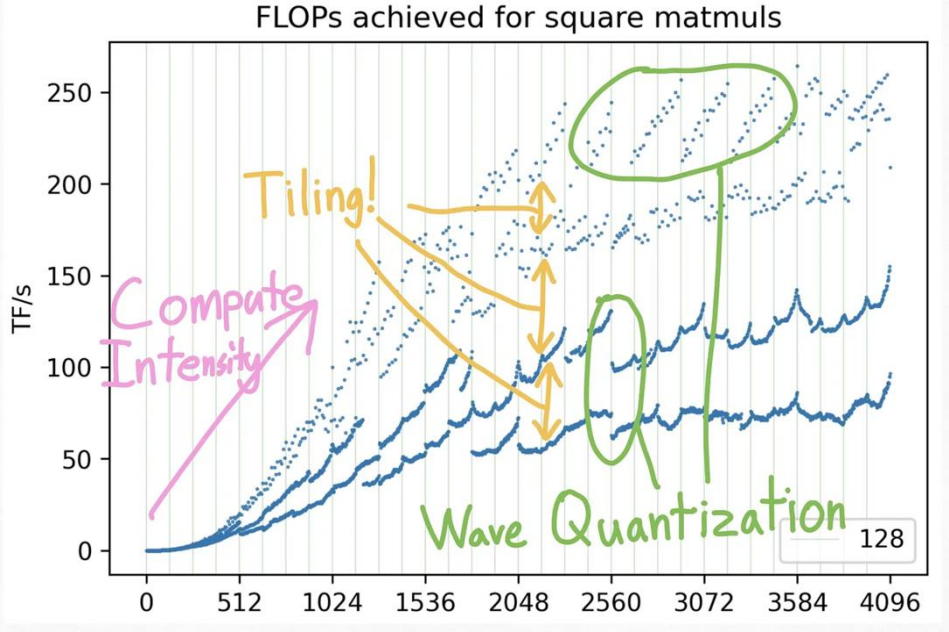
The most dramatic optimization to nanoGPT so far (~25% speedup) is to simply increase vocab size from 50257 to 50304 (nearest multiple of 64). This calculates added useless dimensions but goes down a different kernel path with much higher occupancy. Careful with your Powers of 2.

10:36 AM · Feb 3, 2023 · **1.2M** Views

Why is it *faster* to have bigger matrices?

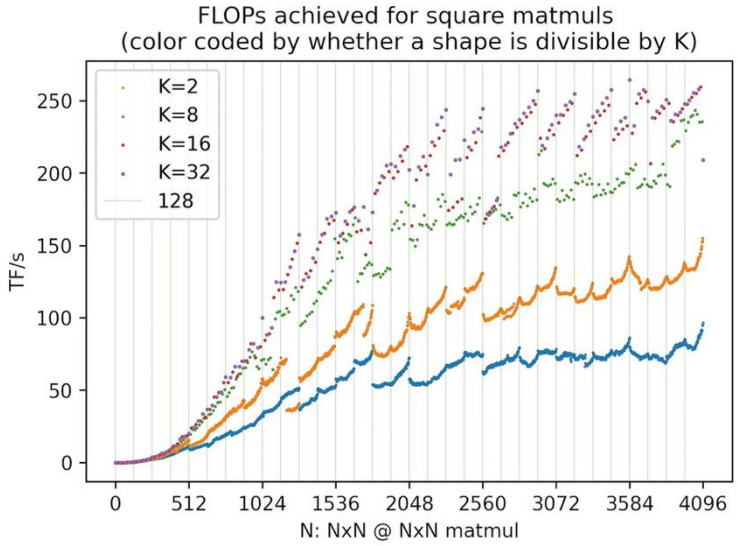
This section is from https://www.thonking.ai/p/what-shapes-do-matrix-multiplications

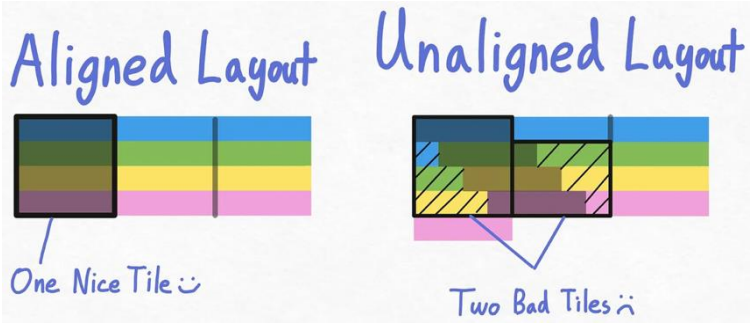
Matrix mystery



We understand some of this (compute intensity, tiling). Let’s take a closer look..

Part 1: tiling





Tiling has a major impact through alignment.

第 5 讲 · 原页 45–47 · 中文讲解

矩阵性能谜题：更大为何有时更快

读懂例子中的“增大词表”

第 45 页截图将词表维度从 50257 扩到 50304，使相关矩阵形状更适合内核选择与硬件对齐，虽然增加少量计算，却可能显著缩短时间。这里的更快必须区分实际 latency 与 TFLOP/s：吞吐提高不一定代表时间减少，只有 F/P_effective 也下降才算该工作更快。对于改变词表的实现，还需确保填充 token 的屏蔽与损失计算保持正确；不能为了对齐悄悄改变任务语义。

将第 **46** 页曲线分成可解释的现象

总体上升可由更高算术强度、更多并行任务和固定启动成本摊薄解释；第 47 页不同余数/对齐曲线的分离与 tile 边界、访存和内核选型相关；周期性掉落还需下一页的 wave 模型。横轴变化可能触发库选择另一套算法，所以某个突变不能只根据图形就唯一归因。原图是固定实验设置下的证据，而不是所有 GPU 的标准曲线。

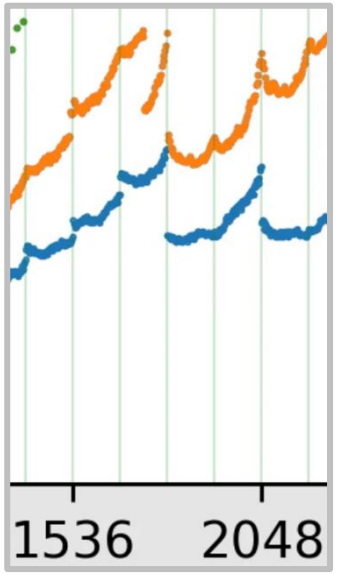
编者补充：如何复现实验而不骗自己

若以后自行测量，应固定 dtype、M/N/K、转置/stride、输出精度与硬件，预热后多次测量，使用正确 GPU 同步或事件计时，避免仅测到 CPU 发射任务的时间。报告原始耗时以及 FLOPs/时间两者。对比 padding 时把额外计算和数据转换也算进端到端成本。本次讲义增强只验证小型 CPU 数值例子，没有声称复现图中的 GPU benchmark。

本节参考（完整书目见讲义末尾）：[slides] · [gemm]

Part 2: wave quantization

What's with the periodic behavior?



This happens at 1792 to 1793 size.

Why? Using a tile size of 256×128, there are

$$\frac{1792}{256} \times \frac{1792}{128} = 7 \times 14 = 98$$

tiles. If we increase this to 1793, we have

$$8 \times 15 = 120$$

tiles.

An A100 has 108 SMs, so it cannot execute all 120

第 5 讲 · 原页 48–48 · 中文讲解

Wave quantization：任务数跨过一波容量

把原页的取整补完整

假设每个输出 tile 为 256×128，方阵输出边长 L，则 tile 数 G=ceil(L/256)·ceil(L/128)。L=1792 时是 7×14=98；L=1793 时是 8×15=120。采用原页的 A100 108 个 SM 且每 SM 此时只承载一个这类 block 的简化模型，一波最多执行 108 块，98 块只需一波，120 块需要第二波处理剩余 12 块。第二波大多数 SM 空闲，造成尾部时间。

编者计算：为什么图会呈周期性波动

若各块耗时相同、一波容量为 W，理想波数是 ceil(G/W)，可用 G/[W·ceil(G/W)] 估计槽位利用率。G=98、W=108 时约 90.7%；G=120 时约 55.6%。随着 G 继续增长，第二波逐渐填满，利用率回升；下一次跨过波边界又可能下降。边界块本来就可能更便宜，实际调度也不是严格同时起停，因而这只是解释机制的近似模型。

不要混淆三种 **quantization**

tile quantization 是单块内部边缘填不满；wave quantization 是全设备最后一波任务填不满；数值 quantization 则是用有限离散数值表示实数。此页的“量化”与 FP8/FP4 无关。实际 W 应由 SM 数乘每 SM 可并发驻留的此类 block 数估计，后者取决于共享内存、寄存器等；不是所有 A100 kernel 都固定 W=108。

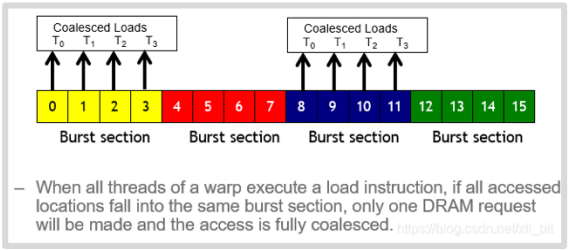
课堂补充 · **01:08:12–01:09:14**

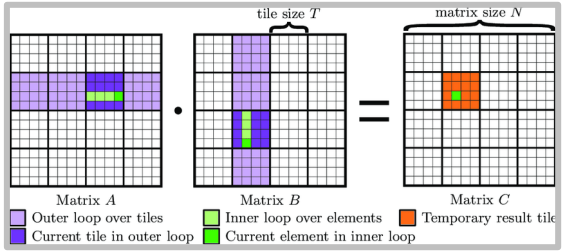
教师用 1792→1793 的 98/120 个 tile 解释曲线周期性下降。这里补上的 ceiling 公式、槽位比例及驻留假设是编者推导，帮助把课堂例子迁移到别的矩阵与设备。

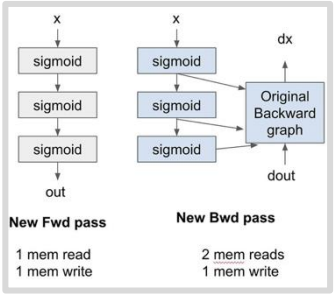
本节参考（完整书目见讲义末尾）：[slides] · [captions] · [gemm]

Recap of part 2: making ML workloads go fast

- Reduce memory accesses
  - Coalescing
  - Fusion
- Move memory to shared memory
  - Tiling
- Trade memory for compute/accuracy
  - Quantization
  - Recomputation

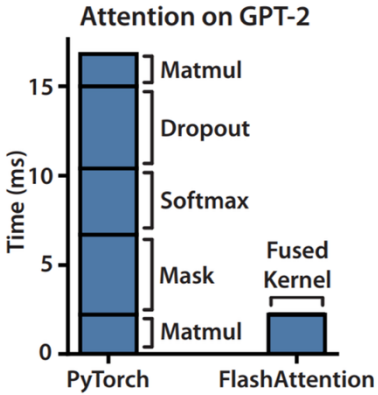




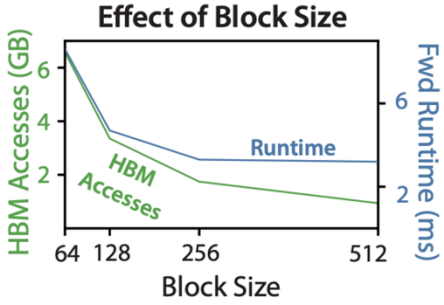


Part 3: Using what we know to understand Flash Attention

Flash attention [Dao et al] dramatically accelerates attention.. But how?



Attention	Standard	FLASHATTENTION
GFLOPs	66.6	75.2
HBM R/W (GB)	40.3	4.4
Runtime (ms)	41.7	7.3



Technique from paper:

We apply two established techniques (tiling, recomputation) to overcome the technical challenge of computing exact attention in sub-quadratic HBM accesses. We describe this in Algorithm 1. The main idea

第 5 讲 · 原页 49–50 · 中文讲解

把性能技巧拼到注意力里

先给各技巧指定职责

第 49 页回顾的几类优化作用不同：coalescing 提高每次传输的有效利用，fusion 消除中间张量往返，tiling 增加片上重用，低精度减少每个数的字节并可能提高矩阵吞吐，重计算用额外算术换保存/读取激活。分析一个算法时，应该能指出具体张量、存储位置和生命周期，而不是只给它贴“优化过”的标签。

FlashAttention 保持什么、改变什么

第 50 页的图和表把标准注意力与 FlashAttention 比较：后者避免把完整注意力得分和概率矩阵反复写到 HBM，以分块和融合重排计算，并在反向重算部分中间量。所谓 exact attention 指保留稠密注意力的数学目标，而不是稀疏化或核近似；浮点运算顺序改变后不保证逐位相同。密集两两交互的算术复杂度仍为 $O(N^2d)$，显存及 IO 改善不能说成“注意力计算变成线性”。

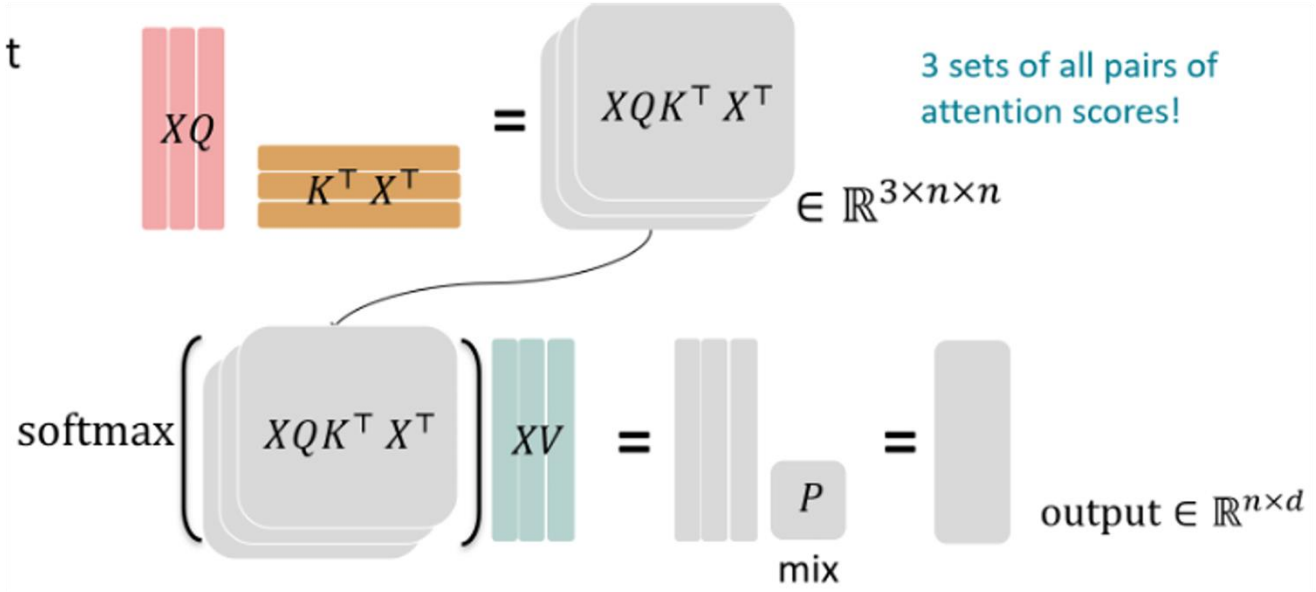
读实验图时保留条件

原图的柱形分解帮助看出 kernel launch、softmax 等非矩阵部分的时间占比，表格展示某个设置的运行时间和内存改善；右图说明 block size 会影响性能。不能把这些倍数直接外推到任意 batch、序列长度、头维或 GPU。下一步是从公式解释为什么能省掉 $N\times N$ 中间矩阵，而不是先记住某个加速倍数。

本节参考（完整书目见讲义末尾）：[slides] · [flash]

Recap of attention computation

Attention computation: 3 matrix multiplies (K, Q, V) with a softmax in between



第 5 讲 · 原页 51–51 · 中文讲解

重新写清注意力的形状与存储

从输入投影到注意力核心

对单个注意力头，设序列长度 N、输入宽度 d_model、头维 d，X 形状为 N×d_model。Q=XW_Q、K=XW_K、V=XW_V，形状均可取 N×d。这三次线性投影与后面的注意力核心不是同一组乘法。核心为 $S=QK^T/\sqrt{d}$，S 为 N×N；逐行 P=softmax(S+mask)，仍为 N×N；O=PV，O 为 N×d。原页“3 matrix multiplies (K,Q,V)”易产生歧义，图中投影与核心应按上述公式分开数。FlashAttention 通常从已经得到的 Q/K/V 开始优化。

编者数字例子：存储怎样被序列长度放大

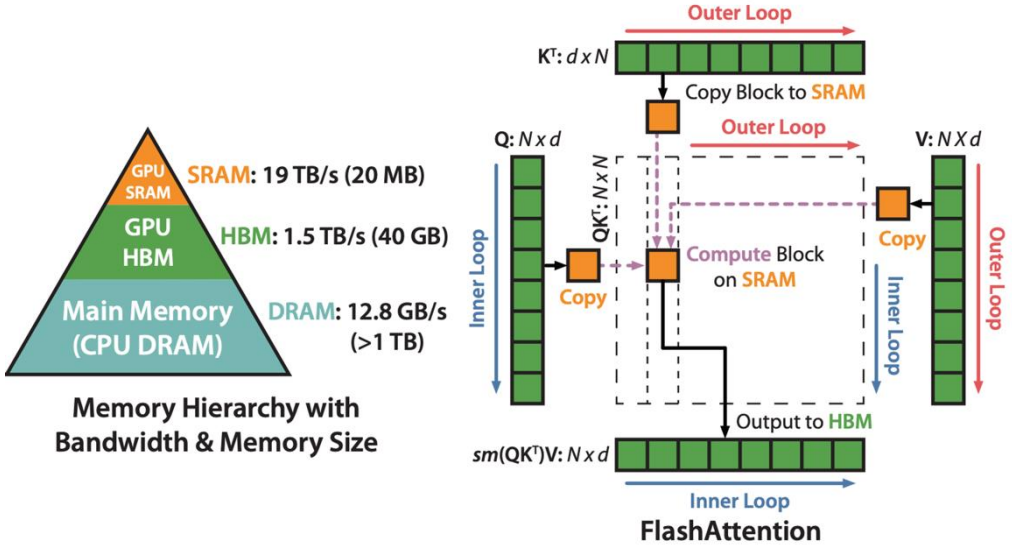
N=4096、d=64 时，单头一个 N×N 的 FP16 矩阵占 $4096^2\times2$ B=32 MiB，而 Q、K、V 各只有 4096×64×2 B=0.5 MiB。若同时物化 S 和 P，再保留反向所需中间量，开销更大；batch 和多头还会乘上对应数量。核心两次矩阵乘法各约 $2N^2d$ FLOPs，总计约 $4N^2d$，尚未计入投影与 softmax。

按行归一化不是按块归一化

某个 query 的输出 $o_i=\Sigma_j p_{ij}v_j$，权重分母包含该 query 所有可访问 key 的指数和。因果 mask 用 $-\infty$ 排除未来位置，排除项指数视作 0。若直接在每个 key tile 内做 softmax 再相加，每个 tile 的概率都会各自和为 1，整体结果就错了；这正是在线 softmax 要解决的跨块依赖。

本节参考（完整书目见讲义末尾）：[slides] · [flash]

Tiling part 1: tiling for the KQV matrix multiply



This figure 1 from the paper is literally just tiling for a KQV matrix multiply..

But what do we do about the softmax?

第 5 讲 · 原页 52–52 · 中文讲解

Attention 分块：难点是归一化跨越所有 key

从原图的存储金字塔看数据生命周期

固定一块 B_r 个 query，把 Q_i 放在片上；依次读取 B_c 个 key 和 value 的块 K_j、V_j。每次算 $S_{ij}=Q_i K_j^T/\sqrt{d}$，形状 B_r×B_c，只需在片上暂存这一小块。若能将它立即转为对输出的贡献并丢弃，就无需保存完整 N×N 的 S 或 P。输出累加器形状为 B_r×d，还需每行少量归一化状态。Q 块、K/V 块、得分块和累加器必须共同适应片上资源。

编者反例：局部 **softmax** 相加为什么错

某个 query 的两块得分各只有一个元素，分别为 0 和 ln(3)，对应标量 value 为 2 和 10。全局 softmax 权重是 1/(1+3)=1/4 和 3/4，输出为 8。若各块各自 softmax，每块权重都是 1，相加得到 12；若再简单除块数得到 6，仍不正确。必须知道每块相对全局的指数总量，不能只知道块内归一化后的输出。

保留充分状态即可继续计算

正确算法保留三类量：已经见过的最大得分 m、相对于 m 的指数和 ℓ、相对于同一 m 的加权 value 和 u。它们分别解决数值稳定、归一化与输出累积。每读一个新块，将旧状态统一换算到新的最大值基准，再合并新块。下一页先推导 m 和 ℓ，第 54 页再加入 u。

本节参考（完整书目见讲义末尾）：[slides] · [flash] · [online]

Tiling part 2: incremental computation of the softmax

From Mikailov and Gimelshein 2018,

Normal softmax

$$y_i = \frac{e^{x_i - \max_{k=1}^{V} x_k}}{\sum_{j=1}^{V} e^{x_j - \max_{k=1}^{V} x_k}} \tag{2}$$

All major DL frameworks are using this safe version for the Softmax computation: TensorFlow

Algorithm 2 Safe softmax

$m_0 \leftarrow -\infty$
for $k \leftarrow 1, V$ **do**
    $m_k \leftarrow \max(m_{k-1}, x_k)$
end for
$d_0 \leftarrow 0$
for $j \leftarrow 1, V$ **do**
    $d_j \leftarrow d_{j-1} + e^{x_j - m_V}$
end for
for $i \leftarrow 1, V$ **do**
    $y_i \leftarrow \frac{e^{x_i - m_V}}{d_V}$
end for

Online softmax

Algorithm 3 Safe softmax with online normalizer calculation

$m_0 \leftarrow -\infty$
$d_0 \leftarrow 0$
for $j \leftarrow 1, V$ **do**
    $m_j \leftarrow \max(m_{j-1}, x_j)$
    $d_j \leftarrow d_{j-1} \times e^{m_{j-1} - m_j} + e^{x_j - m_j}$
end for
for $i \leftarrow 1, V$ **do**
    $y_i \leftarrow \frac{e^{x_i - m_V}}{d_V}$
end for

To keep track of the max, incrementally update the max, and set up a telescoping sum

This lets you compute the softmax tile-by-tile

第 5 讲 · 原页 53–53 · 中文讲解

在线 softmax：换基准而不重扫旧数据

稳定 softmax 的第一步

对一行得分 $x_1,\dots,x_N$，$p_j=\exp(x_j)/\Sigma_k\exp(x_k)$。令 $m=\max_j x_j$，分子分母同乘 $\exp(-m)$，得到 $p_j=\exp(x_j-m)/\Sigma_k\exp(x_k-m)$。最大指数变为 $\exp(0)=1$，减少溢出风险。m 只是数值基准，不改变数学概率。难点是读完全部数据前还不知道全局最大值。

编者推导：旧指数和怎样换到新基准

处理完旧集合 A 后保存 $m=\max(A)$、$\ell=\Sigma_{x\in A}\exp(x-m)$。新块 B 的最大值为 m_B，令 $m'=\max(m,m_B)$。因为 $x-m'=(x-m)+(m-m')$，所以 $\Sigma_{x\in A}\exp(x-m')=\exp(m-m')\cdot\ell$。于是更新为 $\ell'=\exp(m-m')\cdot\ell+\Sigma_{x\in B}\exp(x-m')$。只需旧的两个标量，就能把之前所有元素一起改用新基准；不需要重新加载旧元素。每行分别维护此状态。

编者手算：得分 0、ln2、ln4

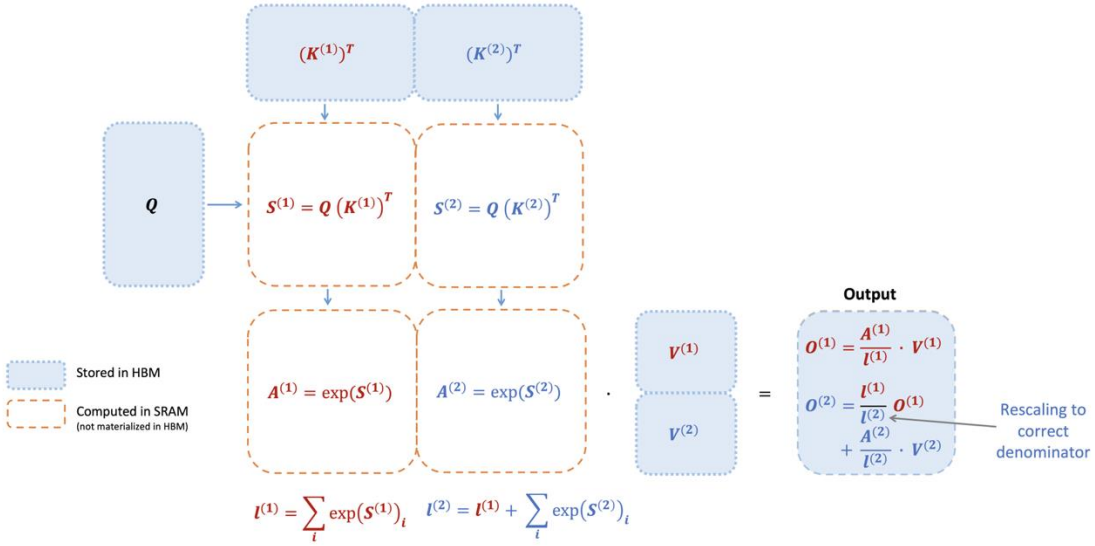
前两个数作为第一块，$m=\ln 2$，$\ell=1/2+1=3/2$；第三个数进入后，$m'=\ln 4$，旧和乘 $\exp(\ln 2-\ln 4)=1/2$，变为 3/4，新项为 1，故 $\ell'=7/4$。最终概率为 $(1/4,1/2,1)/(7/4)=(1/7,2/7,4/7)$。多次换基准时，指数缩放因子相乘会消去中间基准，即 $\exp(m_1-m_2)\exp(m_2-m_3)=\exp(m_1-m_3)$，这就是原页 telescoping 的直觉。

算法边界与课堂补充 · 01:14:15–01:15:37

教师说明遇到更大值时更新最大值并校正累积和。编者补充：初始化可取 $m=-\infty$、$\ell=0$，但首个有效块要避免计算 $-\infty-(-\infty)$；整行被 mask 时需按接口约定特殊处理。在线计算归一化常数，并不意味着只扫一次就能立即输出每个最终 p_j：独立 softmax 输出仍需保留或重读输入；FlashAttention 通过同步累积 P V，避免显式输出所有概率。

本节参考（完整书目见讲义末尾）：[slides] · [captions] · [online]

Putting it all together – the forward pass of flash attention



From Dao 2023, we see

- Tile-wise computation of the inner products, (S)
- Fusion of the exponential operator
- Tile-wise computation of the softmax via the online, telescoping sum trick

(We won't cover the backward pass – but they recompute tile-by-tile..)

第 5 讲 · 原页 54–54 · 中文讲解

FlashAttention 前向：同时更新加权分子

编者推导：从 ℓ 扩展到输出向量

对一个 query，旧状态还保存 $u=\Sigma_{j\in A}\exp(s_j-m)v_j$，u 是 d 维向量。读取新得分块和 value 后，令 $m'=\max(m,\max新得分)$，$\alpha=\exp(m-m')$，$w_j=\exp(s_j-m')$。同时更新 $\ell'=\alpha\ell+\Sigma新块 w_j$，$u'=\alpha u+\Sigma新块 w_jv_j$，最后全部块结束才输出 $o=u/\ell$。因为分子分母始终相对同一个最大值，这与完整 softmax 后乘 V 在实数运算中一致。若实现中保存的是已归一化旧输出 o，则必须改成 $o'=(\alpha\ell o+\Sigma w_jv_j)/\ell'$，不能只乘 α 后直接相加。

编者手算：一次换最大值与输出累积

沿用上一页得分 (0,ln2,ln4)，令标量 value 为 (1,3,5)。第一块状态 $m=\ln2$、$\ell=3/2$，$u=(1/2)\cdot1+1\cdot3=7/2$。读第三个数后 $\alpha=1/2$，$\ell'=7/4$，$u'=(1/2)(7/2)+1\cdot5=27/4$，因此 $o=(27/4)/(7/4)=27/7\approx3.857143$。完整概率 (1/7,2/7,4/7) 的加权平均也为 27/7。对 B_r 行 query，这些标量更新按行广播，u 成为 B_r×d 的矩阵。

按原图串起整个 **kernel**

图中蓝色框表示 HBM 中的输入/输出，虚线区域强调片上分块。一次迭代计算 $Q_i K_j^T$，应用缩放及 mask，更新每行 m、ℓ，立即计算指数权重与 V_j 的块乘积，并更新输出累加器。score 和概率块被消费后丢弃；指数、归约与乘法融合，避免中间量往返 HBM。FlashAttention-2 进一步改进并行划分、减少非 matmul 运算等，本页以其延后归一化的思路解释，不能据一张图声称完整复现了所有版本优化。

反向需要什么：课堂未展开部分的编者补充

常保存 Q/K/V、输出以及每行 log-sum-exp $L=m+\log\ell$ 等小状态；反向按块重新计算 S 和 $P=\exp(S-L)$，避免保存 N×N 概率。梯度也按块累积，不能先构造完整 dP、dS 再声称省内存。若有 dropout，还需重现一致的随机掩码。这是编者对反向重计算的概念补充，课堂没有展开完整内核；末尾自测提供梯度关系供进一步核对。

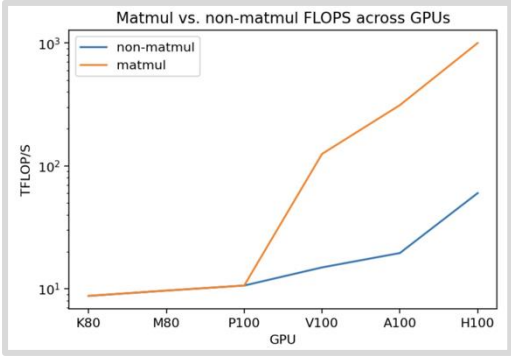
课堂补充 · **01:15:37–01:17:16**

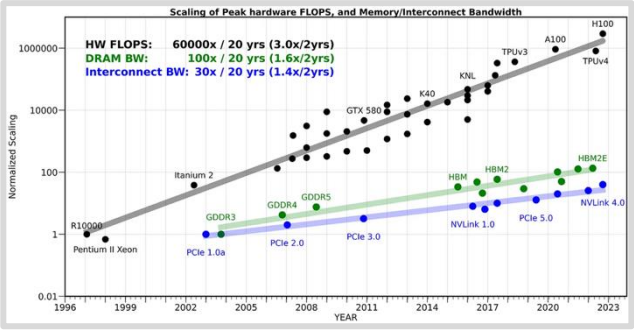
教师依图解释片上得分、指数融合、在线归一化及输出累积，并明确本讲不展开反向，只指出反向逐块重算以避免保存 N^2 激活。低精度路径和累加顺序可能造成微小数值差异，因此正确性验证应按 dtype 和任务设置合理容差；不能把 exact attention 解读为与任意实现逐比特一致。

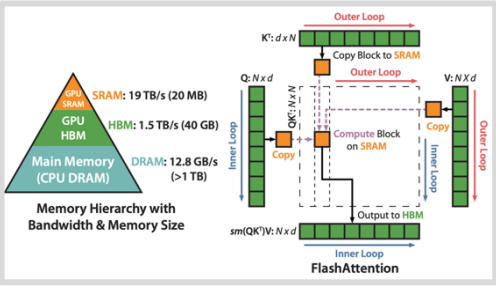
本节参考（完整书目见讲义末尾）：[captions] · [flash] · [flash2]

Recap for the whole lecture

- Hardware powers scale, and low-level details determine what scales or doesnt
- Curent GPU based compute strongly encourages thinking about matmul + data movement
- Thinking carefully about the GPU (coalescing, tiling, fusion) leads us to good performance







第 5 讲 · 原页 55–55 · 中文讲解

把本讲变成一套可执行的分析顺序

从公式到性能解释

面对一个新算子，先写出输入输出形状与数学依赖；再列出每个中间张量的大小、存储位置及保留时长；用一致口径估算 FLOPs 与传输字节并找 roofline 瓶颈；之后看 warp 地址、分支、tile 内重用和全设备任务数。每次提出优化，都说明它减少了哪一种成本，又增加了什么。最后先检验数值正确性，再测量实际时间。

贯穿全讲的一条因果链

GPU 的并行与专用矩阵单元让运算很便宜，但内存容量和带宽没有同比增长；因此先改善访问组织，再尽量留在片上，必要时用重算或低精度换数据流量。FlashAttention 是这条思路的完整实例：数学上靠在线稳定归一化消除跨块障碍，系统上靠分块、融合与重计算降低 IO。它依旧做稠密注意力，却改变了临时结果的生存位置和时间。

继续学习与自测方法

下一讲的 kernel/Triton 会把本讲的抽象原则落到索引、mask、加载、归约与启动配置；并行训练讲则把同样的计算/通信权衡扩展到多设备。先完成末尾自测，尤其亲算在线 softmax 和 tile/wave 两种取整。如果不能说出一个式子在哪个维度归约、一个字节来自哪一级存储，就回到对应原页和解释再推一遍，而不要只记优化名称。

本节参考（完整书目见讲义末尾）：[slides]

自测与参考答案

1. 256 个线程的 block 有几个 warp？它能跨两个 SM 各跑一半吗？

在本讲 NVIDIA 基础模型下有 256/32=8 个 warp。一个 block 由一个 SM 承载；一个 SM 可以承载多个 block，但受寄存器、共享内存和线程等资源限制。warp 数并不表示每周期所有 warp 同时发射。

2. 一张 GPU 的 P=200 TFLOP/s、HBM 带宽 B=2 TB/s。算子 I=20 FLOP/B 时，roofline 吞吐上界和瓶颈是什么？

转折点 P/B=100 FLOP/B，当前低于转折点。带宽屋顶 B I=40 TFLOP/s，故上界 min(200,40)=40 TFLOP/s。增加计算峰值不会改变这个上界，减少 HBM 字节或增加有效带宽才直接有帮助；真实吞吐可能更低。

3. 一百万个 FP32 ReLU 元素的读写量和强度是多少？FP16 呢？

FP32 读 4 MB、写 4 MB，总计 8 MB（十进制），按每元素 1 op 共 10^6 op，强度 1/8 op/B。FP16 总计 4 MB，强度 1/4 op/B。8 B/op 是倒数；低精度在理想纯带宽受限条件下才对应约两倍改善。

4. 每 16 元素一个 E4M3 尺度的 FP4 是什么格式？含组尺度时每元素多少 bit？

这组特征对应 NVFP4；MXFP4 常见对照是每 32 元素一个 E8M0 尺度。16×4+8=72 bit，平均 4.5 bit/元素；未计张量尺度、对齐与其他副本，因此不能据此推算整个训练系统的显存。

5. 三个一元逐元素算子融合能省多少流量？为何也可能变慢？

独立执行理想流量为 6nb，融合后可降到 2nb，中间值不再写回 HBM，启动次数也减少。若融合导致寄存器压力过大并溢出或降低驻留并行度，实际性能可能反而下降，故还需测量。

6. 为什么重计算可以同时省内存并加速？什么时候只省容量却更慢？

如果中间值计算便宜、读写昂贵，重新生成并立即消费能省掉大量 HBM 往返，闲置算力承担了额外计算。若重算的是昂贵且已经计算受限的矩阵乘法，或导致更多同步，就可能增加耗时；激活检查点仍可能因解决显存容量不足而有价值。

7. 行主序 FP32 矩阵宽度为 1024，lane t 读取 A[r,c+t] 与 A[r+t,c] 的地址间隔各是多少？

前者每 lane 相差 4 B，同一条 warp load 地址连续，通常可高效合并；后者每 lane 相差

1024×4=4096 B，分散到不同位置。分析同一条指令的多个线程，不是一个线程跨循环的轨迹。

8. N=1024、T=32 的方阵分块乘法，为什么输入流量下降 32 倍而不是 1024 倍？

共有 $(N/T)^2$ 个输出 tile，每个要处理 N/T 个归约阶段，各阶段装入 $2T^2$ 个输入，总输入元素读取数 $2N^3/T$。一个输入仍被不同输出 tile 读取 N/T=32 次，并不是只从 HBM 读取一次；但相对朴素的 N 次降低了 T=32 倍。实际缓存可以进一步改变物理流量。

9. L=1793、输出 tile=256×128、每波容纳108块，需要几波？这属于哪种量化？

ceil(1793/256)=8、ceil(1793/128)=15，共120块，ceil(120/108)=2波。第二波只有12块，体现wave quantization；边界tile内部还可能有tile quantization浪费。它与FP8数值量化不同，108的容量还依赖每SM驻留一块的假设。

10. 为什么对每个 key tile 分别 softmax 再平均局部输出是错的？

各 tile 的指数和一般不同，局部归一化抹掉了相对权重。得分0和ln3、value为2和10时，全局权重1/4与3/4给输出8，局部输出平均却是6。合并时必须保留最大值、指数和及加权分子。

11. 在线 softmax 已有 m=ln2、ℓ=3/2，新块只有 ln4，新的 m 和 ℓ 是什么？

$m'=\ln 4$，旧状态乘 $\exp(\ln 2-\ln 4)=1/2$，因此 $\ell'=(1/2)(3/2)+\exp(0)=7/4$。不能直接把旧$\ell$加1，因为它仍以旧最大值为基准。

12. 如果上述旧加权分子 u=7/2，新 value=5，最终输出是多少？若维护的是归一化输出又怎么更新？

$u'=(1/2)(7/2)+5=27/4$，输出 $o'=(27/4)/(7/4)=27/7$。若旧状态为 $o=u/\ell$，则 $o'=(\alpha\ell o+\text{新块加权分子})/\ell'$。漏掉旧$\ell$会破坏分子分母的一致基准。

13. FlashAttention 的 exact、低显存与低复杂度各是什么意思？

exact 表示计算稠密注意力的同一数学结果而不使用稀疏或核近似；浮点舍入仍可能不同。低显存来自不物化并保存完整N×N得分/概率，反向逐块重算。密集注意力的算术复杂度仍是$O(N^2d)$，不能把存储与IO改善说成所有计算都变成线性。

14. 如果一行全部被 mask，在线公式有哪些问题？如何处理？

没有有效最大值，直接计算$-\infty-(-\infty)$会得到NaN，分母也可能为0。应依据算子接口约定对全遮罩行

显式处理；例如某些实现返回零输出，但必须与目标语义一致。正常因果自注意力通常保留当前位置，因此每行至少有一个有效项。

15. 编者延伸：给定上游梯度 dO，标准注意力反向的矩阵关系是什么？

令 $S=QK^T/\sqrt{d}$、$P=\mathrm{softmax}(S)$、$O=PV$，则 $dV=P^TdO$，$dP=dO\,V^T$。softmax 的行内导数给 $dS=P\odot(dP-\mathrm{rowsum}(P\odot dP))$，其中行和按列广播；再有 $dQ=dS\,K/\sqrt{d}$、$dK=dS^TQ/\sqrt{d}$。这里 dO/dQ 等表示损失对张量的梯度，d 表示头维。FlashAttention 反向据这些关系分块重算 P 并累积梯度，不显式物化完整 dP/dS。该推导是编者延伸，原课堂未详讲。

参考资料

[course] Stanford CS336 Spring 2026 官方课程与讲次表

https://cs336.stanford.edu/

[slides] 第5讲原始幻灯片，官方仓库，**Spring 2026**

https://github.com/stanford-cs336/lectures/blob/main/lecture_05.pdf

[captions] 公开录像字幕的第三方缓存，未经教师审校；正文给出定位时间

https://jingzechen.github.io/assets/courses/stanford-cs336-s26/lectures/005/captions.source.srt

[cuda] NVIDIA CUDA C++ Programming Guide，官方文档；基础执行/内存模型（旧版指南已归档）

https://docs.nvidia.com/cuda/cuda-c-programming-guide/

[cuda-best] NVIDIA CUDA C++ Best Practices Guide，官方文档；合并访存与共享内存

https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html#coalesced-access-to-global-memory

[tpu-book] How To Scale Your Model：**GPU/TPU** 架构对照技术书，原稿推荐

https://jax-ml.github.io/scaling-book/gpus/

[gemm] NVIDIA Matrix Multiplication Background User Guide，官方文档；**tile** 与 **wave quantization**

https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html#tile-quant

[te] NVIDIA Transformer Engine MXFP8 官方文档；转置方向的独立量化

https://docs.nvidia.com/deeplearning/transformer-engine/features/low_precision_training/mxfp8/mxfp8.html#handling-transposes

[mxfp8] Recipes for Pre-training LLMs with MXFP8，原研究论文，原稿所引 **v2**

https://arxiv.org/html/2506.08027v2

[nvfp4] NVIDIA Introducing NVFP4，官方格式说明；与 MXFP4 对照

https://developer.nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-precision-inference/

[recompute] PyTorch 开发者讨论：Min-cut optimal recomputation，原开发说明，原稿所引

https://dev-discuss.pytorch.org/t/min-cut-optimal-recomputation-i-e-activation-checkpointing-with-aotautograd/467

[online] Milakov 与 Gimelshein：Online normalizer calculation for softmax，原论文

https://arxiv.org/html/1805.02867v1

[flash] Dao 等：FlashAttention，原论文，2022

https://arxiv.org/html/2205.14135v2

[flash2] Tri Dao：FlashAttention-2，作者提供的论文 PDF，2023

https://tridao.me/publications/flash2/flash2.pdf